

SMU – Mediterranean Institute of Technology

ISS Project Report

Smart Campus Classroom Management System

An IoT and Edge-AI Approach to Automated Attendance,
Environmental Monitoring, and At-Risk Detection

Team Members

Mohamed Hedi Ben Jemaa
Ahmed Amine Jallouli
Ali Saadaoui
Abdelhamid Ouertani
Iyed Day

Professor

Donia Driss

Academic Year 2025–2026

May 2026

Abstract

This paper presents the design and implementation of the Smart Campus Classroom Management System, an IoT and edge-AI platform developed at SMU – Mediterranean Institute of Technology. The system addresses three core problems in traditional classroom operation: manual attendance taking (up to 15 minutes per lecture), the absence of real-time environmental monitoring, and the lack of early-warning mechanisms for academically at-risk students.

The solution integrates an ESP32 microcontroller—equipped with temperature, humidity, air-quality, and sound sensors—with a Raspberry Pi 4B edge server that runs all services without cloud dependency. Student attendance is automated via camera-based face recognition using DeepFace/FaceNet 128-dimensional embeddings with cosine-distance matching. Sensor data is streamed over MQTT (Mosquitto broker) to a FastAPI backend and forwarded in real time to a React professor dashboard via WebSocket. AC and lighting are controlled automatically based on configurable thresholds, with manual professor override. Attendance records are synchronised to the Moodle LMS on session completion.

An AI analytics layer, built on a locally hosted Ollama phi3-mini LLM, identifies at-risk students (below 70% attendance) and generates natural-language explanations; a deterministic forecasting pipeline classifies per-course attendance trends and adds LLM-generated prose interpretations. The complete stack is containerised with Docker Compose and was delivered across 22 development phases within a single academic semester. Evaluation demonstrates full elimination of manual attendance overhead, end-to-end sensor latency below 5 seconds, and AI pipeline runtime reduction from approximately 14–16 minutes to 2–3 minutes through batch-query optimisation.

Index Terms

IoT, smart classroom, face recognition, MQTT, FastAPI, edge computing, LLM, at-risk detection, attendance automation, Raspberry Pi, ESP32, Docker, WebSocket, Moodle

I. INTRODUCTION

A. Context and Motivation

The growing adoption of Internet of Things (IoT) technology in education has opened new possibilities for automating routine classroom tasks and improving the quality of the learning environment. Despite this trend, many universities—including SMU – Mediterranean Institute of Technology—continue to rely on manual processes for attendance management and environmental control. Professors spend up to 15 minutes per lecture calling roll, consuming valuable instructional time. Meanwhile, classroom HVAC systems and lighting are operated without any sensor feedback, leaving occupant comfort dependent on manual judgment. Early identification of students at risk of academic failure due to poor attendance remains largely reactive, with intervention arriving after the damage is already done.

The convergence of affordable edge hardware (ESP32, Raspberry Pi 4B), open-source ML frameworks (DeepFace, Ollama), and lightweight message protocols (MQTT) makes it feasible to deploy a fully autonomous classroom management platform at minimal cost, running entirely on the university’s local area network with no cloud dependency.

B. Problem Statement

Three concrete pain points motivate this work:

- 1) **Attendance inefficiency.** Manual roll-call consumes up to 15 minutes per session—time that reduces effective teaching time by 15–20% for a 50-minute lecture. Records are entered manually into the Moodle LMS, creating duplication of effort and data-entry errors.
- 2) **Unmonitored environment.** No real-time sensor data is available to inform HVAC or lighting decisions. Temperature, humidity, CO₂/VOC levels, and occupancy are unknown to both professors and facility managers.
- 3) **No early warning for at-risk students.** Advising staff receive no automated signal when a student’s attendance rate begins to decline. Intervention is reactive rather than preventive, reducing its effectiveness.

C. Project Objectives

The general objective of this project is to design and implement an IoT-based smart classroom management system that automates student attendance via camera-based face recognition, monitors classroom environmental conditions in real time, enables automated and manual control of AC and lighting, provides professors with a live web dashboard, synchronises attendance to the Moodle LMS, and identifies at-risk students and attendance trends using a locally hosted LLM—all running on a Raspberry Pi 4B without cloud dependency.

The six specific objectives are:

- 1) Automate student attendance recording using face recognition.
- 2) Monitor classroom environmental conditions (temperature, humidity, air quality, sound) in real time.
- 3) Enable automated and manual control of AC and lighting via relay actuation.
- 4) Provide professors with a live web dashboard showing attendance, sensor data, and alerts.
- 5) Synchronise attendance records to Moodle automatically on session end.
- 6) Identify at-risk students and forecast attendance trends using AI analytics.

D. Scope and Limitations

The system is scoped to a single classroom room (`room1`) on the university intranet. All services run locally on the Raspberry Pi 4B; there is no internet dependency during normal operation. The academic semester timeline constrained the project to a proof-of-concept deployment with physical hardware; production hardening (HTTPS/WSS, multi-room support) is identified as future work.

Hardware budget constraints led to the selection of the MQ-135 gas sensor, whose output is uncalibrated and used comparatively rather than as an absolute CO₂ reference. The Raspberry Pi 4B's CPU limits face-recognition throughput to 2 fps and phi3-mini inference to 10–30 seconds per student.

E. Report Structure

Section II reviews related work on IoT-based attendance systems, face recognition, and edge AI. Section III presents stakeholder analysis and functional and non-functional requirements. Section IV describes the full system architecture across all subsystem layers. Section V details hardware assembly, firmware, and software implementation. Section VI covers the AI analytics pipelines. Section VII analyses security and privacy considerations. Section VIII documents the testing and validation approach. Section IX presents results and key lessons learned. Section X concludes with a summary of contributions and directions for future work.

II. LITERATURE REVIEW AND RELATED WORK

A. Smart Classroom Systems: State of the Art

Smart classroom systems have evolved significantly over the past decade from simple RFID card readers to multi-sensor IoT platforms. Early approaches such as [1] demonstrated that RFID tags could replace paper registers, reducing attendance time to under two minutes per lecture. However, RFID systems require students to actively tap a reader, introducing a proxy-attendance vulnerability and necessitating additional hardware distributed to each student.

Subsequent work explored QR-code-based attendance [2], which requires only a smartphone but shares the proxy problem and depends on stable Internet connectivity for code rotation. Camera-based approaches eliminate student action entirely: the system identifies occupants passively, closing the proxy loophole and reducing professor workload to zero for the attendance step.

A recurring limitation across these systems is the absence of an integrated environmental monitoring layer and of any AI-driven academic risk analysis. They treat attendance as the sole data point, ignoring classroom conditions that correlate with student engagement and comfort.

B. Face Recognition in Attendance Systems

Face recognition for attendance has been studied extensively using conventional CNN-based pipelines [3], [4]. FaceNet [3] introduced the 128-dimensional embedding space with triplet-loss training, enabling sub-centimetre face-to-face distance computation via cosine similarity. DeepFace [4] unifies multiple model backends (Facenet, ArcFace, VGG-Face) behind a single Python API, significantly reducing integration effort.

Our system uses DeepFace with the Facenet backend. Enrollment generates a 128-dimensional float32 embedding averaged from up to five reference images per student; recognition applies cosine-distance thresholding ($d < 0.40$) at 2 fps. This threshold is tighter than the DeepFace default (0.60) to reduce false-positive attendance records on a shared-CPU Raspberry Pi 4B.

Prior work [5] showed that Facenet inference on an RPi 4B reaches 1–3 fps under normal lighting, consistent with our empirical observations. The primary tradeoff is latency: the 2 fps ceiling means a 30-student class requires up to 15 seconds to scan all faces in one frame pass, motivating the snapshot-per-cycle attendance model described in Section V-E.

C. IoT Protocols for Educational Environments

Three lightweight messaging protocols are commonly evaluated for IoT sensor networks: MQTT [6], CoAP [7], and HTTP polling. MQTT's publish-subscribe architecture decouples producers (ESP32 sensor nodes) from consumers (backend services) without requiring the sensor to know the consumer's address. Its binary framing and persistent TCP connection produce less per-message overhead than HTTP, and its broker-mediated delivery survives transient client disconnections through configurable quality-of-service (QoS) levels.

QoS 0 (fire-and-forget) is used throughout this system. For sensor readings published every 5 seconds, an occasional dropped message causes at most a 10-second gap in the time-series record—acceptable for environmental monitoring. Upgrading to QoS 1 would guarantee delivery at the cost of increased broker state and acknowledgement round-trips, which is unnecessary on a local network with sub-1 ms latency.

TABLE I
COMPARISON OF CLASSROOM MANAGEMENT APPROACHES

Feature	RFID [13]	QR-Code [2]	CNN Face [5]	Cloud IoT [14]	This Work
Attendance method	RFID card	QR scan	Face rec.	Face rec.	Face rec.
Proxy-attendance proof	No	No	Yes	Yes	Yes
Environment monitoring	No	No	No	Yes	Yes
LMS synchronisation	No	No	No	Partial	Yes
AI risk detection	No	No	No	No	Yes
Cloud-free deployment	Yes	No	Yes	No	Yes
Real-time dashboard	No	No	No	Yes	Yes
Open-source stack	Partial	Partial	Partial	No	Yes

D. Local LLM Inference at the Edge

Recent advances in model quantisation have made it feasible to run billion-parameter language models on commodity CPU hardware. The phi3-mini model [8] (3.8 B parameters) from Microsoft achieves strong reasoning quality with a 4k-token context window, suitable for generating analytical prose from structured attendance data. Ollama [9] provides a local model server with a simple REST API, eliminating the need for an API key or cloud subscription.

The rationale for a local LLM over a cloud API is threefold: (i) student attendance data should not leave the institution’s network boundary; (ii) cloud API costs scale with usage and require internet access during every pipeline run; and (iii) a local model is always available, decoupling the AI layer from internet connectivity.

The primary limitation is inference speed: on the RPi 4B CPU, phi3-mini requires 10–30 seconds per student explanation. The at-risk pipeline batches all enrolled at-risk students into a single pipeline run with one LLM call per student, and a Redis-based cooldown (10-minute TTL) prevents redundant re-computation on each page refresh [10].

E. LMS Integration

Moodle [11] is the dominant open-source Learning Management System in higher education. It exposes a comprehensive REST API authenticated via token, enabling programmatic creation and update of grade and attendance records. Automated Moodle sync has been demonstrated in related work [12] as an effective way to eliminate manual data entry and ensure that the official academic record reflects real-time classroom events.

Our implementation calls the Moodle REST API on session end, writing presence/absence records for each enrolled student. A Redis retry queue handles transient Moodle unavailability, with retry jobs running on a 10-minute scheduler cycle.

F. Summary and Positioning

Table I compares this system with four categories of existing approaches across seven distinguishing features. Our system is the only one in this comparison that combines passive face recognition with real-time environmental monitoring, local LMS synchronisation, and on-premises AI-driven at-risk detection—all without cloud dependency.

III. REQUIREMENTS ANALYSIS

A. Stakeholder Analysis

Four stakeholder groups were identified during the requirements elicitation phase:

- **Professors (primary users).** Professors start and end sessions, view live sensor and attendance data, manually adjust attendance records, control AC and lighting, and acknowledge alerts. Their primary goal is to recover the 15 minutes per lecture lost to manual roll-call.
- **Students (indirect beneficiaries).** Students benefit from more accurate attendance records and improved classroom comfort resulting from automated environmental control. They do not interact with the system directly.
- **Administration (secondary users).** Administrators have access to aggregated attendance and at-risk data across all courses, enabling resource optimisation and compliance reporting. They can trigger AI pipeline recomputation on demand.
- **Advising department (data consumers).** Academic advisers use the at-risk dashboard to initiate early interventions before students miss the withdrawal deadline.

B. Functional Requirements

Table II lists the functional requirements grouped by subsystem.

TABLE II
FUNCTIONAL REQUIREMENTS

ID	Subsystem	Requirement
FR01	Attendance	System shall recognise enrolled students from camera frames during an active session.
FR02	Attendance	System shall record attendance status (present/absent/late) per session per student.
FR03	Attendance	System shall allow professors to manually adjust any attendance record.
FR04	Attendance	System shall fill absent status for all enrolled students not detected at session end.
FR05	Attendance	System shall implement bidirectional present $\leftrightarrow$ absent evaluation every cycle.
FR06	Attendance	System shall never overwrite manually adjusted records.
FR07	Environment	System shall publish temperature and humidity every 5 seconds.
FR08	Environment	System shall publish air-quality (ppm proxy) every 5 seconds.
FR09	Environment	System shall publish binary sound detection every 5 seconds.
FR10	Environment	System shall persist sensor readings to a time-series database.
FR11	Control	System shall allow professors to set AC to on/off/auto via dashboard.
FR12	Control	System shall automatically activate AC when temperature exceeds 28 °C in auto mode.
FR13	Control	System shall allow professors to set lighting to on/off/auto via dashboard.
FR14	Dashboard	System shall display live sensor readings with less than 5 seconds end-to-end latency.
FR15	Dashboard	System shall display current session attendance roster in real time.
FR16	Dashboard	System shall display active alerts with acknowledgement capability.
FR17	Dashboard	System shall operate in demo mode when backend is unreachable.
FR18	Alerts	System shall alert when temperature exceeds 28 °C or falls below 22 °C.
FR19	Alerts	System shall alert when air-quality exceeds 500 ppm.
FR20	Alerts	System shall alert when the ESP32 device goes offline.
FR21	LMS Sync	System shall synchronise attendance to Moodle on session end.
FR22	LMS Sync	System shall retry failed Moodle syncs on a scheduled basis.
FR23	AI	System shall identify students below 70% attendance as at-risk.
FR24	AI	System shall generate natural-language explanations for at-risk students using a local LLM.
FR25	AI	System shall classify per-course attendance trends deterministically.
FR26	AI	System shall generate LLM-assisted prose interpretations for attendance forecasts.

C. Non-Functional Requirements

Table III presents the non-functional requirements.

TABLE III
NON-FUNCTIONAL REQUIREMENTS

ID	Category	Requirement
NF01	Performance	Sensor update round-trip (ESP32 $\rightarrow$ dashboard) shall be ≤ 5 seconds.
NF02	Performance	API response time for non-AI endpoints shall be ≤ 500 ms under normal load.
NF03	Performance	At-risk pipeline shall complete in ≤ 5 minutes for up to 10 at-risk students.
NF04	Availability	System shall operate without internet access during normal classroom use.
NF05	Security	Professor authentication shall use bcrypt-hashed passwords and JWT tokens.
NF06	Security	Role-based access control shall restrict admin endpoints to admin-role accounts.
NF07	Security	Face embeddings shall not be stored as raw images; only 128-d BYTEA arrays are persisted.
NF08	Scalability	Architecture shall support extension to multiple rooms by changing the <code>ROOM_ID</code> environment variable and replicating the ESP32 node.
NF09	Resilience	MQTT bridge shall reconnect with exponential backoff on broker restart.
NF10	Resilience	Frontend shall activate demo mode with client-side mock data after 8 seconds without WebSocket data.
NF11	Resilience	AI pipelines shall set <code>ollama_reachable=false</code> and continue without crashing when Ollama is unavailable.
NF12	Usability	All professor-facing interactions (start session, relay control, alert ack) shall require ≤ 2 clicks.

D. Use Case Overview

Figure 1 illustrates the primary actors and use cases. The Professor actor initiates all classroom-level interactions; the Admin actor adds AI pipeline control; the ESP32 Node autonomously publishes sensor data; and Moodle receives attendance records at session end.

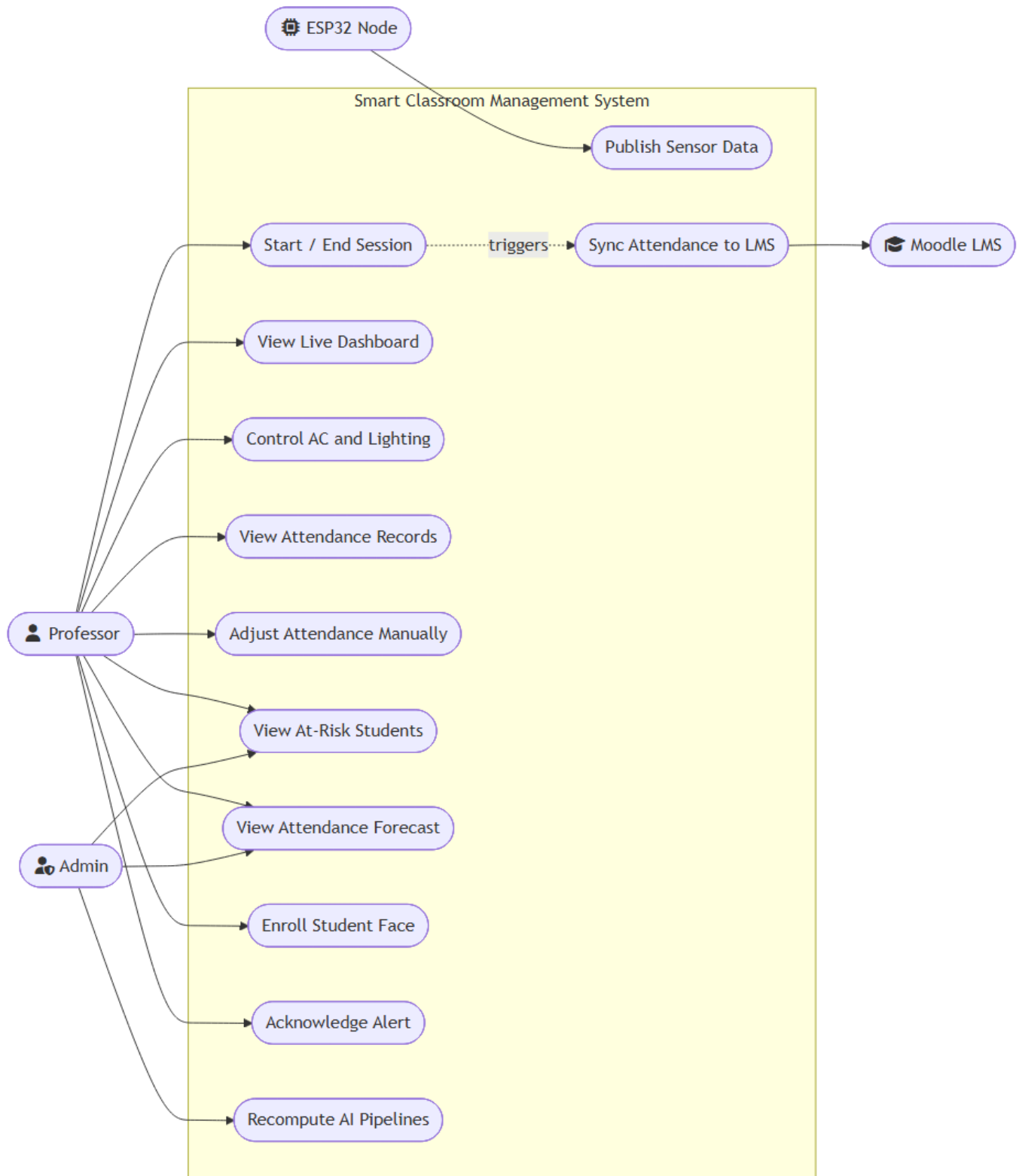


Fig. 1. Use case diagram showing four actors and twelve use cases.

The critical use case sequence for a normal lecture is: (1) Professor starts session → (2) Face recognition loop activates → (3) Attendance records update live on dashboard → (4) Professor ends session → (5) Final absent fill runs → (6) Attendance syncs to Moodle.

E. Constraints

Hardware constraints. The Raspberry Pi 4B (4 GB RAM) is the sole compute node. Face recognition is limited to 2 fps; phi3-mini inference runs at 10–30 seconds per student on CPU.

Budget constraints. Total hardware cost is approximately \$150 USD, including the RPi 4B, ESP32, sensors, relay module, and camera. See Table XV in Section V-B for the detailed bill of materials.

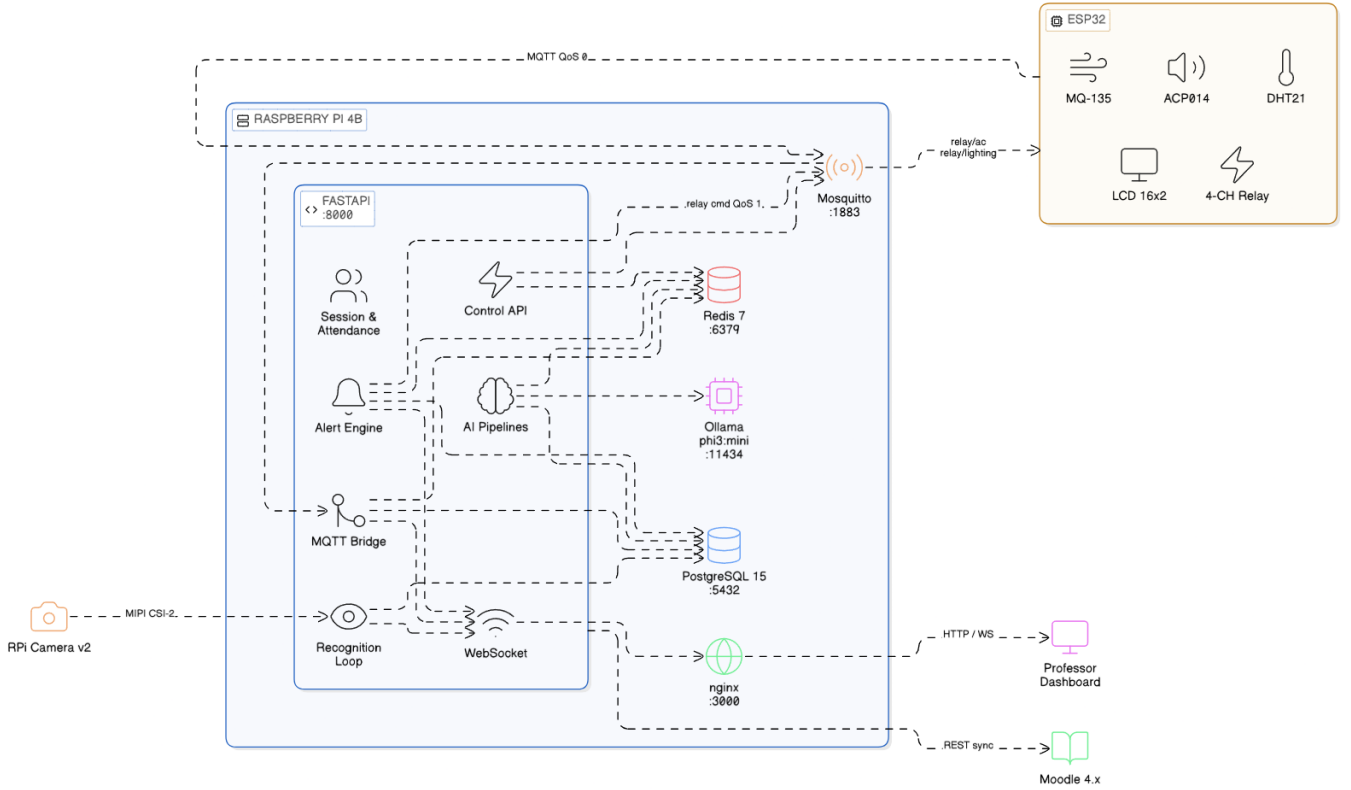
Timeline constraint. The project was delivered within one academic semester (22 weeks, 22 phases). Features deferred to future work include multi-room support, HTTPS/WSS, and quantised GGUF model inference.

IV. SYSTEM ARCHITECTURE

A. Architecture Overview

The system follows a *hybrid modular monolith with event-driven messaging* architecture. The backend is a single FastAPI process; however, it is internally structured around three decoupled event channels: an MQTT ingestion pipeline, a WebSocket fan-out layer, and an on-demand AI pipeline bus. This avoids the operational overhead of microservices while preserving the loose coupling that makes each subsystem independently testable.

Figure 2 shows the five-tier logical view: IoT devices at the edge, a Mosquitto message broker, the FastAPI backend with its service groups, the React frontend, and external integrations (Moodle, Ollama).



eraser

Fig. 2. Full system layered architecture: IoT → Edge → Backend → Frontend → External.

Table IV summarises the technology choices for each layer, and Table V lists each subsystem's primary responsibility.

TABLE IV
TECHNOLOGY STACK SUMMARY

Layer	Technology	Key Libraries
Firmware	Arduino C++	PubSubClient, DHT, LiquidCrystal_I2C
Backend	Python 3.11	FastAPI, SQLAlchemy 2.0, aiomqtt, APScheduler, httpx
Database	PostgreSQL 15	Alembic (migrations)
Cache	Redis 7	redis-py async
AI	Ollama phi3-mini	httpx AsyncClient
Face Rec.	DeepFace	OpenCV, FaceNet, numpy
Frontend	React 18	Vite, TailwindCSS, Recharts, axios
Infra	Docker Compose	nginx, eclipse-mosquitto:2

TABLE V
SUBSYSTEM RESPONSIBILITIES

Subsystem	Responsibility
IoT (ESP32)	Sense environment; publish MQTT; actuate relays
Edge (RPI)	Run all services locally; no cloud dependency
MQTT Broker	Message bus between ESP32 and backend
Backend (FastAPI)	Orchestration, REST + WebSocket APIs, business logic
Real-time Layer	Fan-out of sensor/attendance/alert events to browsers
AI/Insights Layer	At-risk detection, forecasting, SQL analytics
Frontend (React)	Professor dashboard; demo mode when offline
Data Layer	PostgreSQL (durable) + Redis (ephemeral live state)
External	Moodle LMS sync; Ollama LLM inference

B. IoT Layer — ESP32 Firmware

1) *Sensor Acquisition*: The ESP32-WROOM-32 runs an Arduino sketch that reads four sensors on a 5-second cycle: the DHT21 temperature/humidity sensor (GPIO 4), the MQ-135 air-quality ADC (GPIO 34, ADC1 channel 6), and the ACP014 digital sound sensor (GPIO 35). GPIO 34 and 35 are input-only pins, which avoids conflicts with the WiFi subsystem that disables ADC2.

2) *MQTT Publishing*: Each reading is serialised as a JSON payload `{value, unit, ts}` and published to the corresponding MQTT topic at QoS 0. A heartbeat message (`status` topic, every 30 seconds) allows the backend to detect device offline events via a TTL-based Redis presence key.

3) *Relay Actuation and Local Auto-Control*: The ESP32 subscribes to `classroom/room1/relay/ac` and `classroom/room1/relay/acAutoMode`. On receipt, it drives the corresponding GPIO output through a logic-level converter (3.3 V $\leftrightarrow$ 5 V) to the 4-channel opto-isolated relay module. An on-device auto-control flag (`acAutoMode`) implements temperature thresholding independently of the backend—a deliberate redundancy that maintains AC control if MQTT connectivity drops.

Figure 3 shows the complete GPIO wiring.

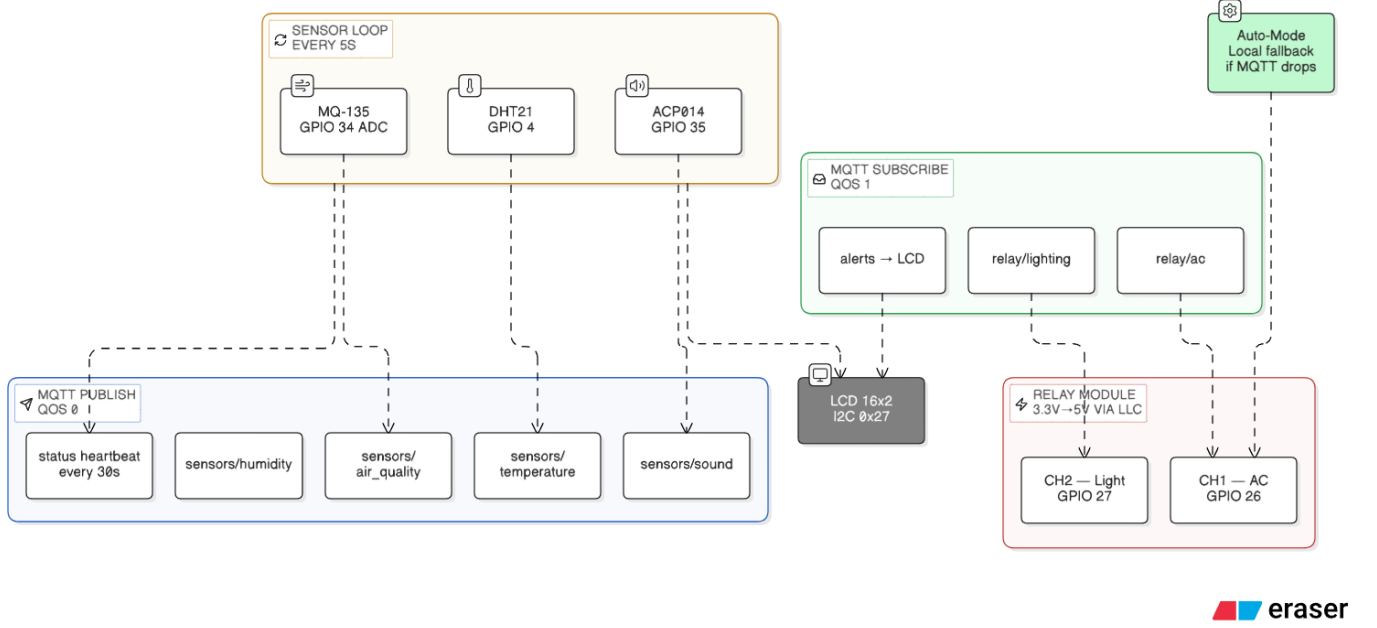


Fig. 3. ESP32 IoT node: sensor inputs, logic-level converter, relay module, and LCD 16×2 wiring.

C. Edge Layer — Raspberry Pi 4B

The Raspberry Pi 4B (4 GB RAM) acts as the central edge hub, running all Docker services on a single host. In production, the RPi also hosts the RPi Camera Module v2, which feeds the face-recognition loop at 2 fps via `OpenCV VideoCapture(0)`.

When `FACE_RECOGNITION_ENABLED=false` (Docker/dev mode), a stub recognition loop fires synthetic attendance events using randomly sampled enrolled students, exercising the full bidirectional present↔absent evaluation logic without a physical camera.

Figure 4 illustrates the runtime topology on the Raspberry Pi 4B, including all Docker service ports and the WiFi link to the ESP32 node.

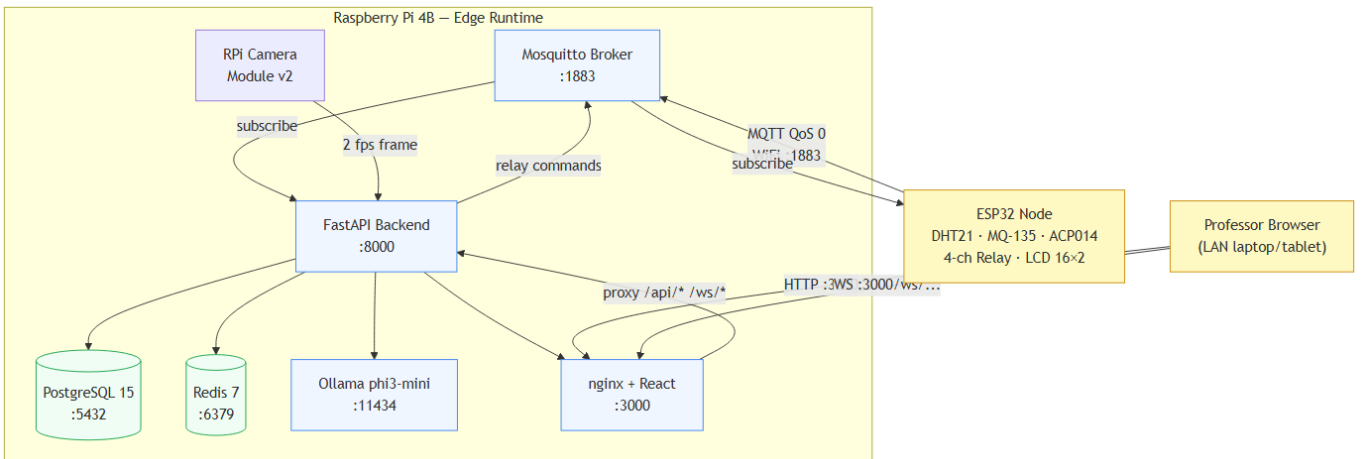


Fig. 4. RPi 4B production runtime topology: all services and their communication paths to the ESP32 classroom node and professor browser.

D. Communication Layer — MQTT

1) *Topic Architecture*: The Mosquitto 2 broker hosts a single hierarchical topic tree under the prefix `classroom/{room_id}/`. Table VI defines all topics, directions, and payload schemas.

TABLE VI
MQTT TOPIC SCHEMA

Topic suffix	Dir.	Payload / Period
sensors/temperature	ESP→RPi	{value,unit,ts} · 5 s
sensors/humidity	ESP→RPi	{value,unit,ts} · 5 s
sensors/air_quality	ESP→RPi	{value,unit,ts} · 5 s
sensors/sound	ESP→RPi	{value:0 1,ts} · 5 s
status	ESP→RPi	{online:true,ts} · 30 s
relay/ac	RPi→ESP	{action:on off auto}
relay/lighting	RPi→ESP	{action:on off auto}
alerts	RPi→ESP	{type,value} → LCD

2) *Subscription Strategy*: The backend subscribes using the wildcard pattern `classroom/+ /sensors/#` to receive sensor messages from any room ID and any sensor sub-topic in a single subscription. A separate subscription on `classroom/+ /status` handles heartbeat messages. ESP32 nodes subscribe to exact topics for relay commands.

Figure 5 shows the complete topic tree.

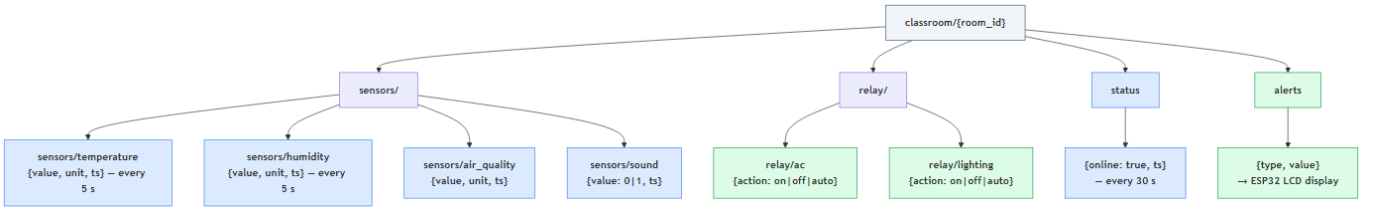


Fig. 5. MQTT topic tree under `classroom/{room_id}/`. Blue nodes are ESP32-published; green nodes are backend-published.

E. Backend — FastAPI Application

The backend is organised into seven logical service groups. Figure 6 shows their connections.

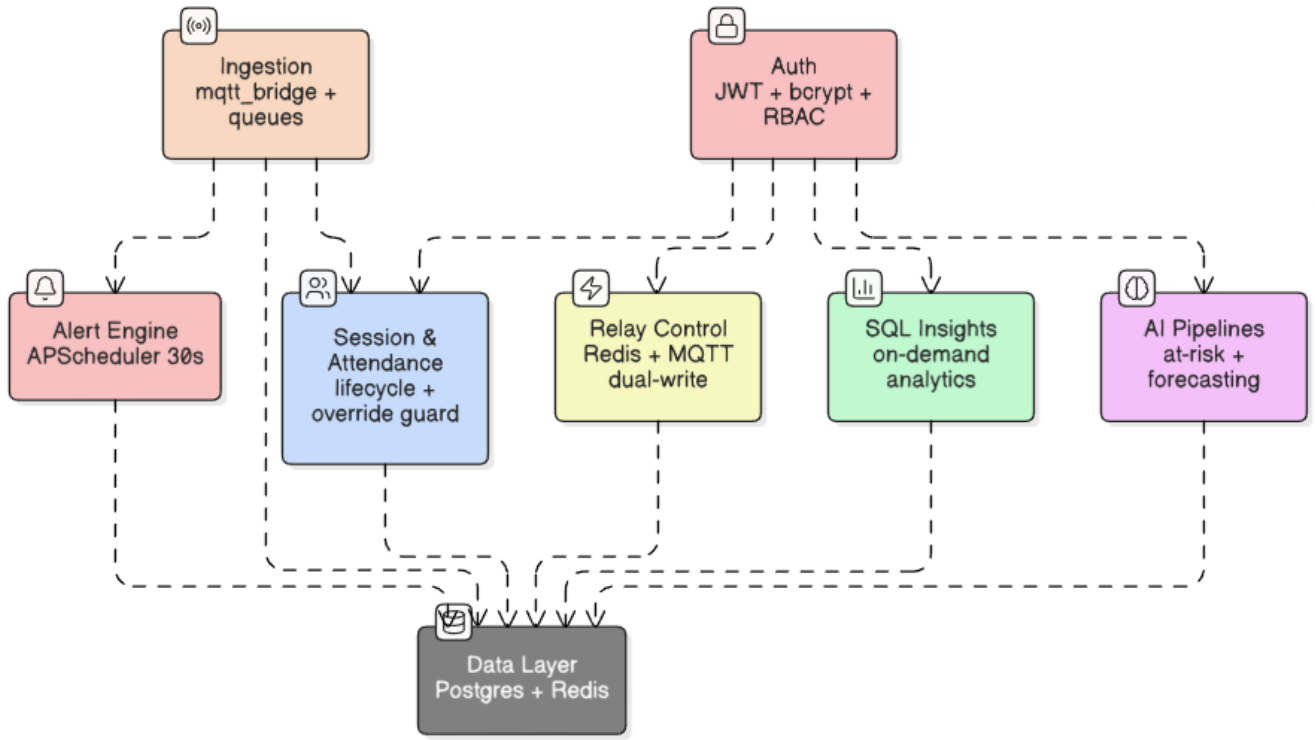


Fig. 6. Backend internal module groups and their data-flow connections.

Group 1 — Ingestion & Real-time (`mqtt_bridge.py`). A persistent `aiomqtt.Client` subscriber loop connects to Mosquitto with exponential backoff. On each sensor message: (i) Redis SET with 5-minute TTL, (ii) fire-and-forget PostgreSQL write, (iii) non-blocking put to `sensor_event_queue`. Three `asyncio.Queue` instances decouple producers from WebSocket consumers.

Group 2 — Session & Attendance (`api/sessions.py`, `recognition_loop.py`). Session lifecycle: POST `/api/sessions/` activates the recognition loop; POST `/api/sessions/{id}/end` stops it, runs the final evaluation cycle, and triggers Moodle sync. Manual adjustments via PATCH `/api/attendance/{id}` set `adjusted_by` to prevent overwrite by the recognition loop.

Group 3 — Control (`api/control.py`). Relay commands simultaneously write Redis (instant dashboard state) and publish MQTT (device actuation). Redis is the authoritative relay state; PostgreSQL is not used for relay records.

Group 4 — Alert Engine (`services/alert_engine.py`). An `AsyncIOScheduler` job runs every 30 seconds, reads the Redis sensor cache, evaluates thresholds (Table VII), and upserts `alerts` rows. Deduplication prevents duplicate alerts for the same unacknowledged condition.

Group 5 — Insights & Analytics. Pure SQL analytics: weekly trend, day/hour heatmap, comfort score, AC effectiveness, temperature versus attendance scatter, air-quality versus sound correlation. Role-filtered: professors see only their courses.

Group 6 — AI Pipelines (Section VI).

Group 7 — Auth (`api/auth.py`). JWT (HS256), bcrypt passwords, role-based access control; `REQUIRE_AUTH=false` in development.

TABLE VII
AUTO-CONTROL RULES

Sensor	Condition	Action
Temperature	> 28°C + AC in auto	Turn AC ON
Temperature	< 22°C + AC in auto	Turn AC OFF
Air quality	> 500 ppm	Alert (no relay)
Sound	Silent > 30 min (active session)	Attendance anomaly alert
Device	Heartbeat TTL expired	Device-offline alert

TABLE VIII
APSCHEDULER JOB INVENTORY

Job	Interval	Purpose
check_thresholds	30 s	Sensor threshold evaluation, relay auto-control, alert creation
retry_moodle_sync	10 min	Re-send up to 10 queued session IDs to Moodle
mock_sensor_publish	5 s	(MOCK_MODE) Synthetic MQTT sensor messages
mock_heartbeat	30 s	(MOCK_MODE) Synthetic status heartbeat

Table IX lists the primary REST endpoints.

TABLE IX
PRIMARY REST API ENDPOINTS

Method	Path	Description
GET	/api/sensors/latest	Latest sensor values (Redis)
GET	/api/sensors/history	Time-range history (PG)
POST	/api/sessions/start	Create + start session
POST	/api/sessions/{id}/end	End session, sync Moodle
POST	/api/control/ac	Set AC relay state
POST	/api/control/lighting	Set lighting relay state
GET	/api/sessions/{id}/attendance	Attendance roster
PATCH	/api/attendance/{id}	Manual adjustment
POST	/api/students/{id}/enrollment	Face enrollment
GET	/api/at-risk	At-risk list + trigger pipeline
GET	/api/forecasting	Forecast list + trigger pipeline
WS	/ws/classroom/{room_id}	Live event stream

F. Real-Time Layer — WebSocket

Figure 7 shows the asyncio fan-out mechanism. Three background drain tasks loop indefinitely, pulling events from their respective queues and calling `connection_manager.broadcast(room_id, event)`. The `ConnectionManager` maintains a `dict[room_id → set[WebSocket]]` for room-scoped delivery.

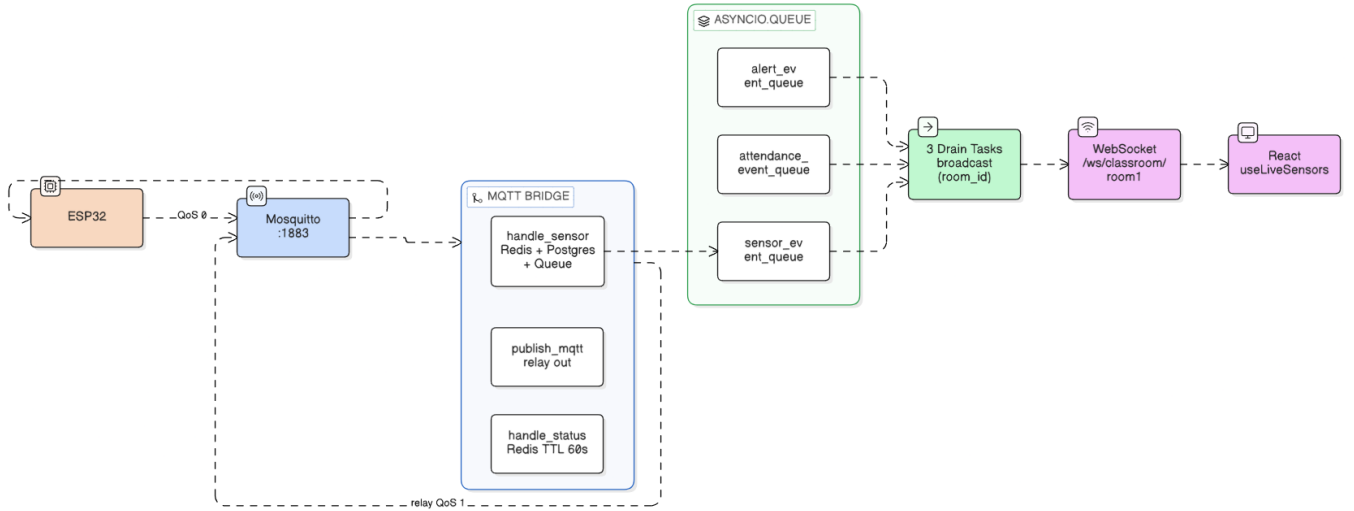


Fig. 7. Event fan-out: three asyncio queues drain independently into the WebSocket broadcast layer.

On connect, the server immediately sends a `snapshot` message containing current Redis sensor values, relay states, and device-online status, ensuring the dashboard is never blank on page load. A 30-second server-side keepalive (ping/pong) detects stale connections. The frontend reconnects with exponential backoff (3 s initial, 30 s cap). Table X defines all WebSocket message types.

TABLE X
WEBSOCKET MESSAGE TYPES (SERVER → CLIENT)

Type	Trigger	Payload fields
snapshot	On connect	sensors, relay, device_online
sensor	MQTT ingestion	room_id, sensor_type, value, unit
attendance	Recognition match	session_id, student_id, name, confidence, status
alert	Threshold breach	alert_type, room_id, message, value
ping	30 s keepalive	—

G. Data Layer

1) *PostgreSQL Schema*: The database contains nine tables. Figure 8 shows the entity-relationship diagram.

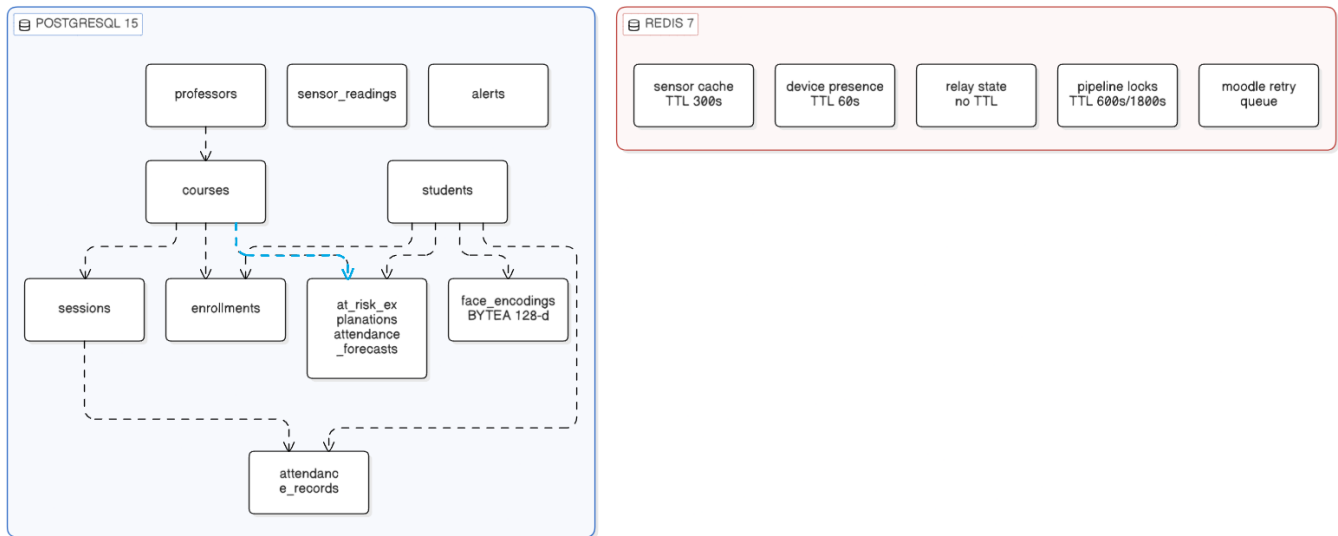


Fig. 8. Entity-relationship diagram for all nine database tables.

Table XI summarises each table’s purpose and key design decisions.

TABLE XI
DATABASE SCHEMA SUMMARY

Table	Key Design Points
professors	UUID PK, bcrypt password, role ENUM (professor/admin)
students	UUID PK, UNIQUE institutional student_id
courses	UUID PK, UNIQUE code, nullable professor FK
sessions	display_status computed (not stored) to reflect live state
attendance_records	adjusted_by guard: recognition loop never overwrites manual changes
face_encodings	128-d float32 BYTEA; no raw images stored
sensor_readings	Time-series; queried for history and AI pipeline sensor averages
at_risk_explanations	UNIQUE student_id; per_course_data as JSONB
attendance_forecasts	UNIQUE course_id; VARCHAR(30) for classification, not PG ENUM

Key schema decisions: display_status is computed at query time because session liveness changes over time without a write. per_course_data is JSONB because it is always read/written as a unit with the parent row. trend_classification uses VARCHAR(30) instead of a PostgreSQL ENUM to avoid non-transactional ALTER TYPE ADD VALUE DDL on schema evolution.

2) *Redis Key Patterns*: Table XII defines all Redis key patterns, their semantics, and TTL behaviour.

TABLE XII
REDIS KEY PATTERNS

Key pattern	TTL	Semantics
classroom:{r}:sensor:300 s	300 s	Latest sensor value for type t
classroom:{r}:online:60 s	60 s	Device presence (heartbeat-renewed)
classroom:{r}:relay:None	None	Relay state; persists across restarts
at_risk:pipeline:lock:600 s	600 s	Prevents concurrent pipeline runs
forecast:pipeline:lock:800 s	800 s	Prevents concurrent forecast runs
Moodle retry list	None	Session IDs awaiting Moodle sync

H. Frontend — React Dashboard

The frontend is a single-page application (React 18, Vite, React Router v6). Two context providers supply global state: `AuthContext` (JWT token, professor profile) and `SensorContext` (wrapping the `useLiveSensors` hook that manages the WebSocket connection and demo-mode fallback).

The design system separates tokens from component classes: `tokens.css` defines all CSS custom properties (colours, spacing, radius, typography); `index.css` uses those variables in component class rules. TailwindCSS v3 utility classes coexist but cannot resolve CSS custom properties at build time, making the token file split necessary.

Figures 9–11 show the three primary professor-facing views.

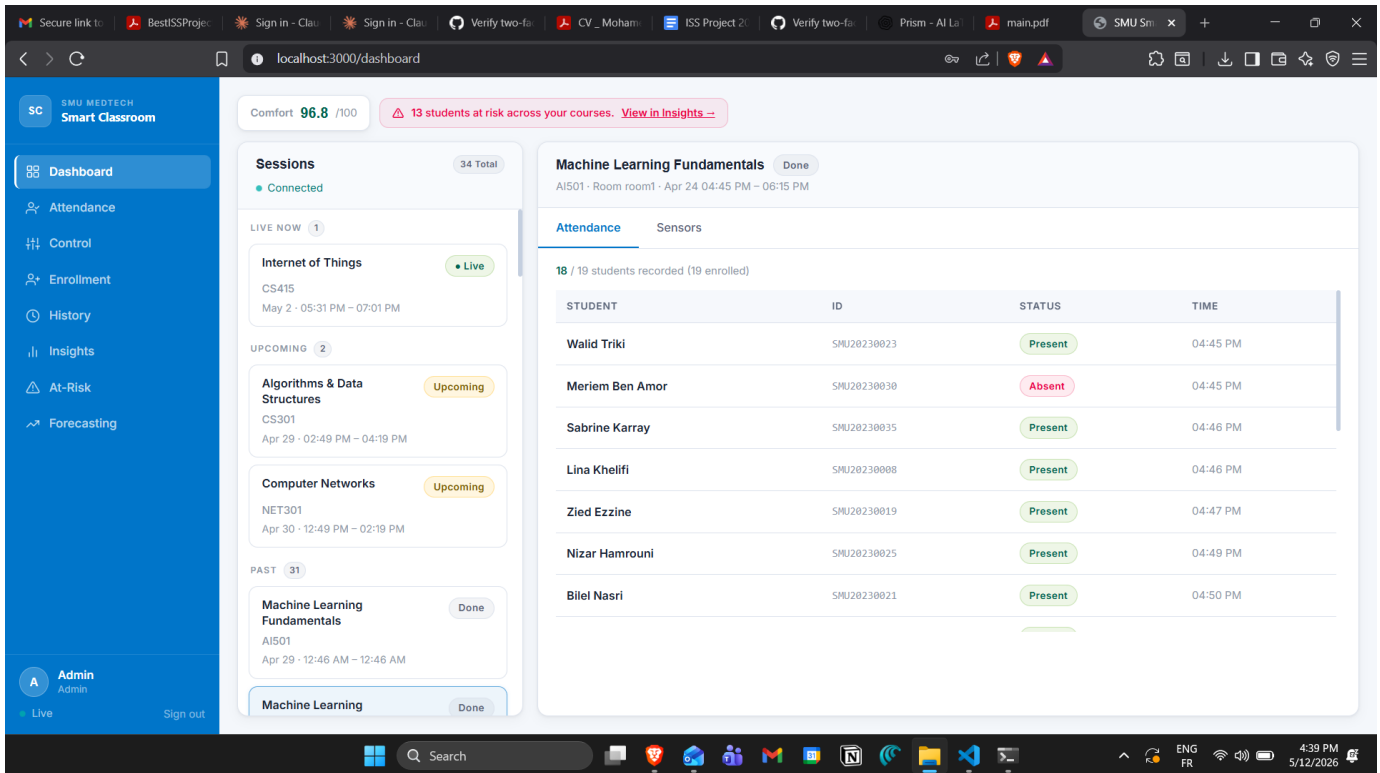


Fig. 9. Dashboard page: live sensor sparklines, session control, and relay status cards.

Attendance

SE302 — 4/20/2026

Mark absent Export CSV

COURSE	PRESENT	ABSENT	LATE	TOTAL
SE302	16	2	3	21

STUDENT	ID	STATUS	DETECTED AT	NOTE
Ons Ferchichi	SMU20230022	Absent	05:00 PM	
Self Saidi At Risk	SMU20230029	Present	05:00 PM	
Walid Triki At Risk absent 2x	SMU20230023	Absent	05:00 PM	
Sarra Belhaj	SMU20230004	Present	05:01 PM	
Skander Jebali	SMU20230015	Present	05:04 PM	
Amira Dridi At Risk absent 2x	SMU20230016	Present	05:04 PM	

Fig. 10. Attendance page: per-student status table with manual adjustment capability.

Control

TEMPERATURE 23.9 °C HUMIDITY 52 % CO₂ 283 ppm

Air Conditioning Off OFF

ON OFF AUTO

Lighting Off OFF

ON OFF AUTO

AUTO-MODE RULES

- AC turns on automatically when temperature exceeds 28 °C
- AC turns off automatically when temperature drops below 22 °C
- Air quality alert fires when CO₂ exceeds 500 ppm

Action log

No actions yet

Commands you send will appear here.

Fig. 11. Control page: relay toggle controls with live sensor summary cards.

I. External Integrations

Moodle 4.x. The optional Moodle integration calls the Moodle REST API (token-authenticated) on session end to write attendance records. Failures are enqueued in Redis; the alert engine retries up to 10 items every 10 minutes. Moodle runs as an optional Docker Compose profile to avoid adding 600 MB of container weight to the default development environment.

Ollama / phi3-mini. The Ollama service exposes a REST API at `http://ollama:11434`. On backend startup, an async background task checks for the phi3:mini model via `GET /api/tags` and pulls it (`POST /api/pull`) if absent—a non-blocking operation that takes approximately 2 GB of download on first run. Both AI pipelines share a single `httpx.AsyncClient` to reuse TCP connections and avoid per-call TCP handshake overhead.

J. Deployment Architecture

Figure 12 shows the Docker Compose service dependency graph. PostgreSQL and Redis must pass health checks before the backend starts. The frontend starts after the backend is running, so that the nginx proxy target is always reachable.

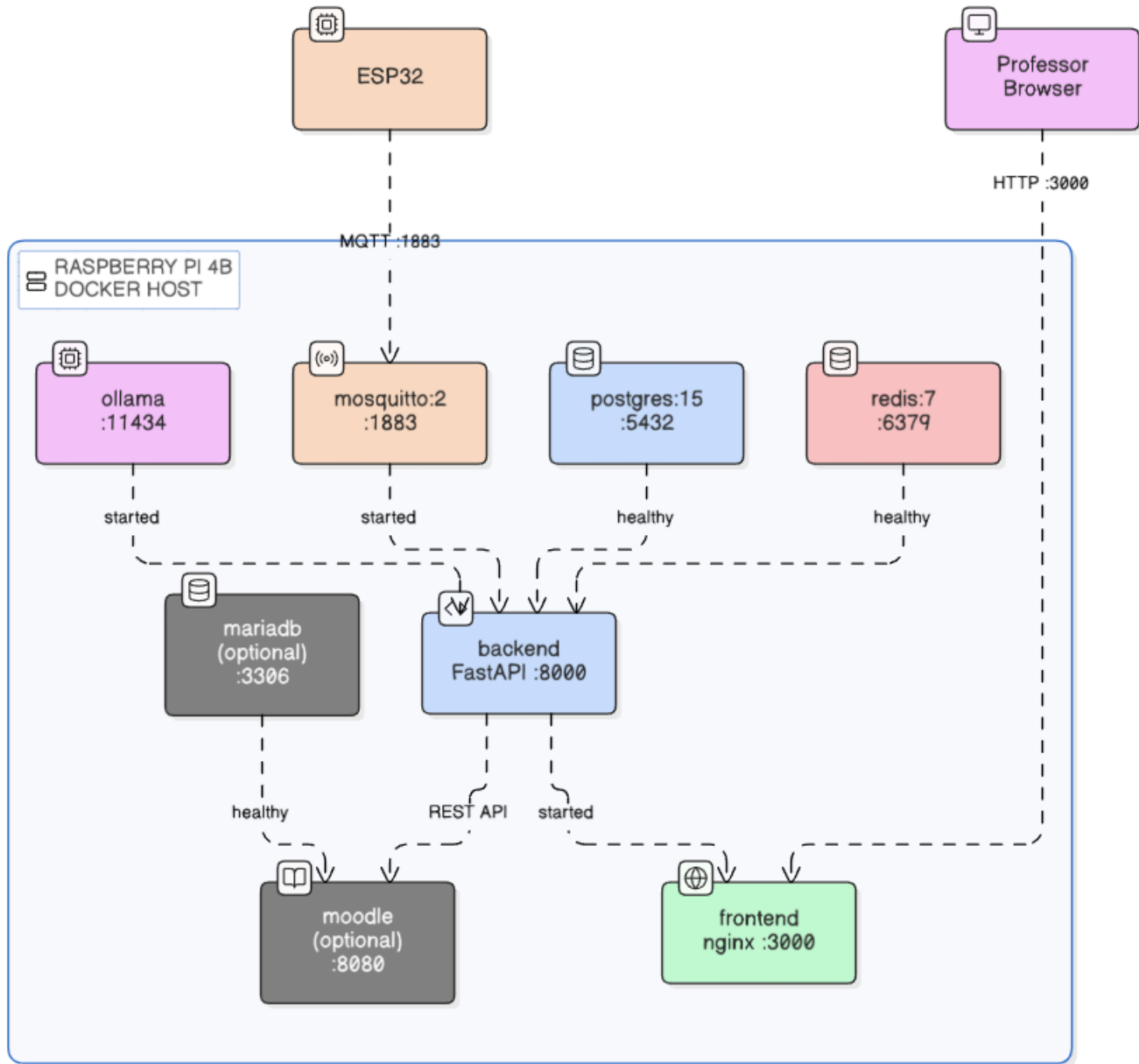


Fig. 12. Docker Compose service graph showing dependency order and container image sources.

Alembic migrations run automatically in the FastAPI lifespan context (`upgrade head`) before any API routes are registered, ensuring schema currency on every container restart without a manual migration step.

K. Key Architectural Decisions

Table XIII captures the most significant non-obvious architectural choices and their rationale.

TABLE XIII
KEY ARCHITECTURAL DECISIONS

Decision	Rationale
FastAPI over Flask	Native async for WebSocket + MQTT concurrent I/O
All services on RPi	Avoids latency, cost, and internet dependency
asyncio-mqtt → aiomqtt	paho-mqtt v2 broke the asyncio-mqtt API
DeepFace stub in Docker	TensorFlow adds ~600 MB; causes OOM on dev laptops
Redis for sensor state	Sub-millisecond read on page load; no PostgreSQL hit
display_status computed	Liveness is time-relative; storing it would require continuous writes
tokens.css split	TailwindCSS v3 cannot resolve CSS vars at build time
Glassmorphism removed	GPU-expensive; caused rendering artifacts in Chrome
Inline SVG icons	No icon-library dependency; only 10–12 icons needed
per_course_data JSONB	Always read/written as a unit; avoids join
Redis TTL not deleted on completion	Prevents re-run on every page refresh (natural cooldown)
1 LLM call/student	N+1 calls: 14–16 min; batched: 2–3 min
Deterministic classification	LLM labels are unreliable for structured values; delta math is deterministic
VARCHAR(30) not PG ENUM	ALTER TYPE ADD VALUE is non-transactional in PostgreSQL
Marker rows (< 3 sessions)	Without a row, the frontend poll condition never resolves
No Redis cooldown in webcam endpoint	last_posted_status in the laptop script is the dedup layer; backend cooldown silently dropped transitions

V. IMPLEMENTATION

A. Development Methodology and Task Management

The project followed an Agile-inspired iterative delivery model, organised into 22 numbered phases spanning hardware assembly, firmware, backend services, frontend, Docker deployment, and AI analytics. Each phase had a clear definition of done and was tracked on a shared Notion board. Figure 13 shows the task board at the time of final submission.



Fig. 13. Notion project board: phase breakdown, assignee tracking, and completion status.

Table XIV shows the 22 delivery phases and their completion status.

TABLE XIV
PHASE COMPLETION STATUS

Ph.	Description	Done
0	Project scaffolding, Docker Compose, env setup	✓
1	ESP32 firmware — sensors + MQTT publish	✓
2	Backend foundation — FastAPI, DB models, MQTT bridge	✓
3	Face recognition service + enrollment API	✓
4	Session management + attendance engine	✓
5	Control API + alert engine	✓
6	Moodle sync service	✓
7	WebSocket live streaming	✓
8	React frontend	✓
9	Integration testing + documentation	✓
10	Full Docker deployment — nginx, Mosquitto wiring	✓
11	Mock data fallback — backend + frontend demo mode	✓
12	Raspberry Pi setup runbook	✓
13	Demo data seed script	✓
14	JWT auth, professors table, role-based filtering	✓
15	Docker image slim — stub mode, self-migrating seed	✓
16	Professor Dashboard — sensor sparklines, session list	✓
17	Dashboard bug fixes — enrolled count, key resets	✓
18	Full frontend visual redesign — CSS token system	✓
19	At-Risk Explanation — Ollama/phi3-mini pipeline	✓
20	At-Risk pipeline performance optimisation	✓
21	LLM-Assisted Forecasting + Recharts trend chart	✓
22	Snapshot-per-cycle attendance; laptop mode conflict fix	✓

B. Hardware Assembly

Table XV lists the bill of materials. All components interface to either the ESP32 or the Raspberry Pi 4B.

TABLE XV
HARDWARE BILL OF MATERIALS

Component	Role
Raspberry Pi 4B 4 GB	Central hub; runs all Docker services and camera
RPi Camera Module v2	8 MP camera input for face recognition
ESP32 WROOM-32	Sensor node + relay controller via MQTT
DHT21 (AM2301)	Temperature and humidity (GPIO 4)
MQ-135	Air quality / CO ₂ proxy (ADC GPIO 34)
ACP014 Sound Sensor	Binary occupancy/noise detection (GPIO 35)
4-Channel Relay	AC (CH1), lighting (CH2), spare (CH3/4)
LCD 16×2 I2C	Local room status display (I2C GPIO 21/22)
Logic Level Converter	3.3 V ↔ 5 V: ESP32 to relay
USB-C 5 V 3 A PSU	Raspberry Pi power supply
32 GB MicroSD	RPi OS and Docker volume storage
Breadboard + wires	Prototyping connections

The critical wiring consideration is the logic-level converter between the ESP32 (3.3 V logic) and the relay module (5 V TTL inputs). Driving the relay directly from the ESP32 GPIO would result in unreliable switching, as the relay's opto-isolator threshold is nominally 5 V. The converter's low-voltage side connects to ESP32 GPIO 26–27 (relay CH1/CH2), and the high-voltage side feeds the relay IN pins.

Figure 14 shows the Raspberry Pi 4B with the camera module attached.



Fig. 14. Raspberry Pi 4B with Camera Module v2 mounted. The RPi hosts all backend services and the face recognition loop.

C. Firmware Development (ESP32)

The Arduino sketch (`classroom_node.ino`) implements a polling loop: every 5 seconds it reads all sensors, constructs JSON payloads, and publishes each to its MQTT topic. Configuration parameters (WiFi SSID/password, MQTT broker IP,

room ID, threshold values) are centralised in `config.h` so deployment to a new classroom requires only editing that file and re-flashing.

MQTT reconnection is handled by a `reconnect()` function called from `loop()` whenever `client.connected()` returns false, with a 5-second back-off delay between attempts. The LCD displays live temperature, humidity, air quality, and sound status on alternating lines, providing local room status without requiring the professor to access the dashboard.

D. Backend Development

Project structure. The backend follows a flat service architecture under `backend/app/`: `api/` for route handlers, `services/` for business logic, `models/` for SQLAlchemy ORM models and Pydantic schemas, and top-level `main.py`, `config.py`, `database.py`, and `redis_client.py`.

Database initialisation. Alembic auto-migration runs on startup via a subprocess call (`alembic upgrade head`) inside the FastAPI lifespan context manager. This ensures the schema is always current without a separate migration step.

MQTT bridge. The `aiomqtt` bridge runs as an `asyncio.Task`, maintaining a persistent subscription to `classroom/+sensors/#` and `classroom/+status`. Exponential back-off reconnection (initial 1 s, max 60 s) handles Mosquitto restarts.

Environment variables. Table XVI lists the primary configuration variables.

TABLE XVI
PRIMARY ENVIRONMENT VARIABLES

Variable	Description and Default
DATABASE_URL	PostgreSQL async connection string
REDIS_URL	Redis connection URL
MQTT_BROKER_HOST	Mosquitto hostname (default: mosquitto)
ROOM_ID	Active room identifier (default: room1)
MOCK_MODE	true: synthetic MQTT without ESP32
FACE_RECOGNITION_ENABLED	true: real camera + DeepFace (RPI only)
SECRET_KEY	JWT signing key — must be replaced in production
REQUIRE_AUTH	false in dev; true in production
AT_RISK_THRESHOLD	Attendance rate below which a student is at-risk (default: 0.70)
ATTENDANCE_CYCLE_DURATION	Seconds per scan-evaluate cycle (default: 60)
OLLAMA_BASE_URL	Ollama service URL (default: http://ollama:11434)
FORECAST_WINDOW	Past sessions included in trend (default: 8)

E. Face Recognition Pipeline

Figure 15 shows the complete enrollment and recognition flow.

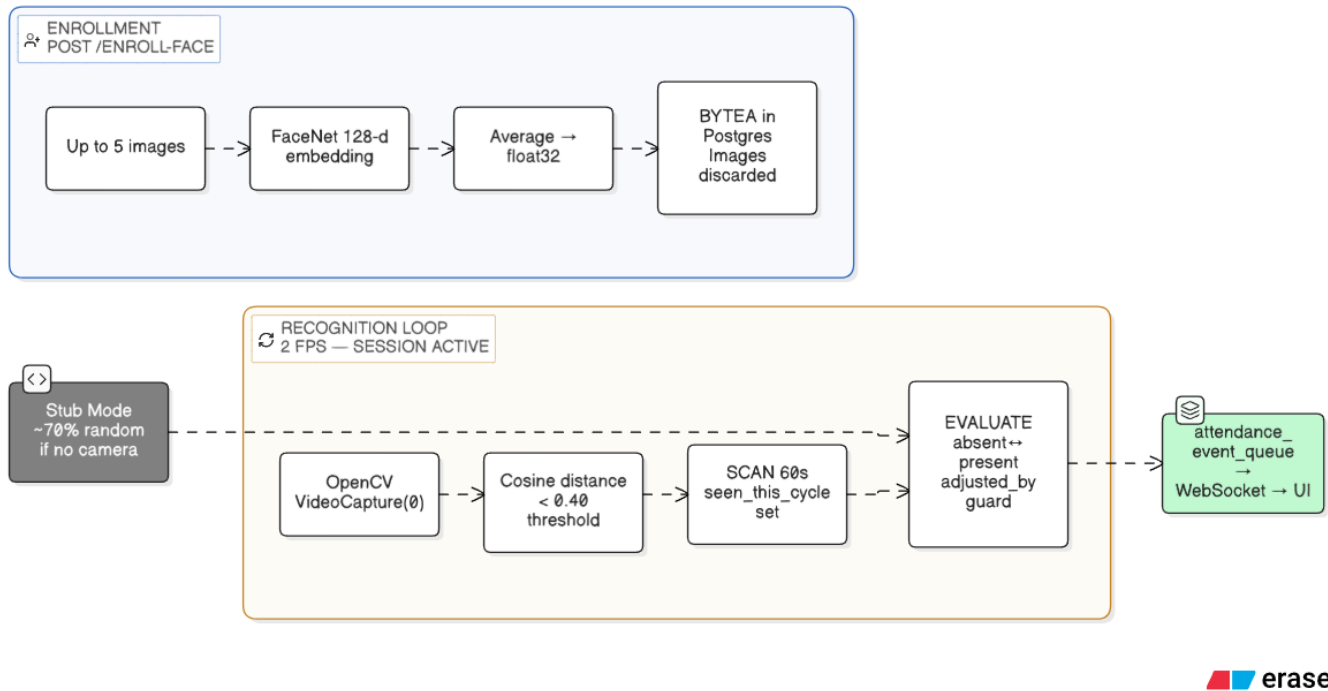


Fig. 15. Enrollment and recognition pipeline: from image upload to attendance record creation.

Enrollment. The professor uploads up to five reference images per student via the Enrollment page (Figure 16). Each image is passed to DeepFace’s `represent()` function with the FaceNet backend, producing a 128-dimensional float32 embedding. The per-image embeddings are averaged and stored as a BYTEA column in the `face_encodings` table. No raw images are retained.

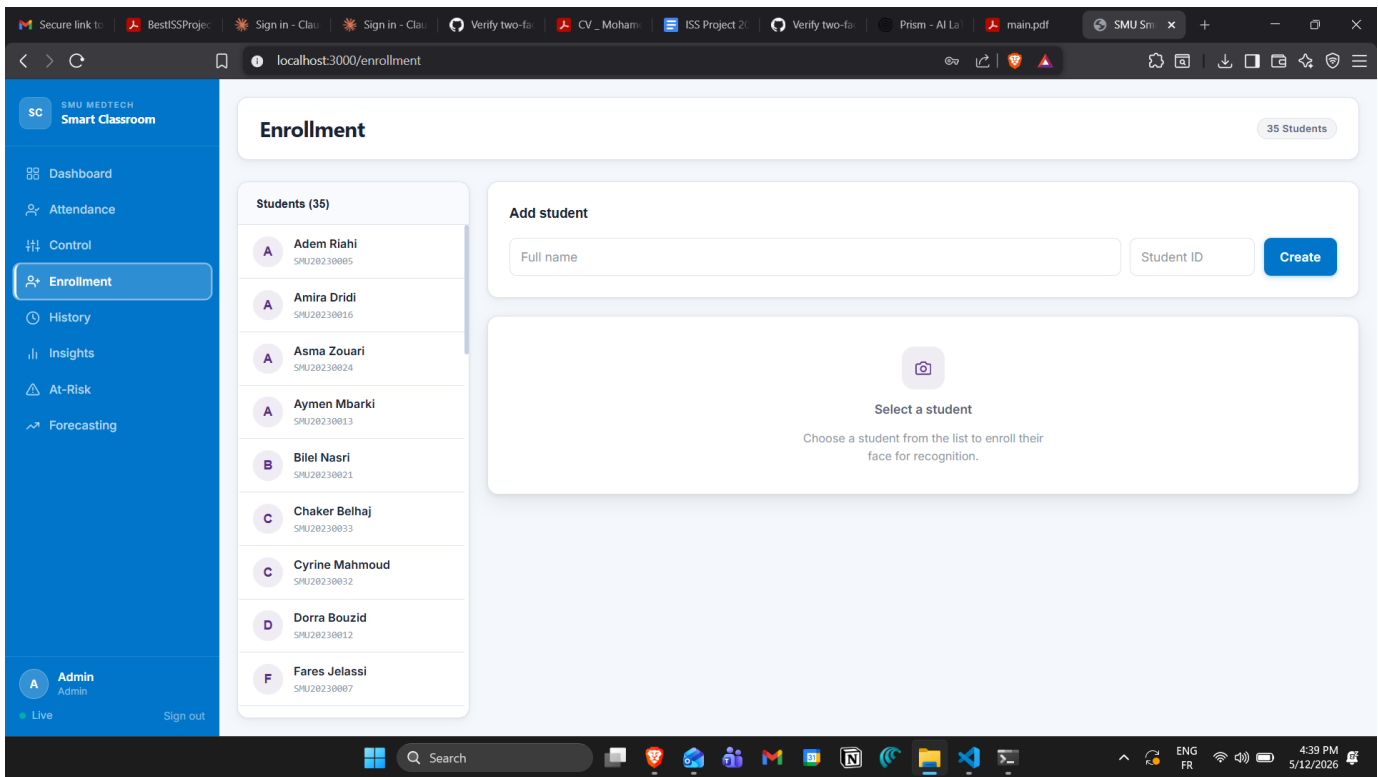


Fig. 16. Enrollment page: student registration and face image upload.

Recognition loop — snapshot-per-cycle model. While a session is active, a background `asyncio.Task` runs the recognition loop:

- 1) **SCAN phase** (duration: `ATTENDANCE_CYCLE_DURATION` seconds at `RECOGNITION_FPS` fps): capture frames, compare each detected face embedding against all stored embeddings using cosine distance. Any student whose distance falls below 0.40 is added to `seen_this_cycle`.
- 2) **EVALUATE phase** (single transaction): for every enrolled student — if in `seen_this_cycle`: mark present; if not: mark absent. Records where `adjusted_by` IS NOT NULL are never touched.

This bidirectional model captures students who leave mid-session, which the original one-shot model missed.

F. Frontend Development

The frontend is scaffolded with Vite and React 18. The `useLiveSensors` hook manages the WebSocket lifecycle, the demo-mode watchdog, and message dispatch to their respective React state slices. If no WebSocket data arrives within 8 seconds of connecting, the hook activates a client-side mock generator and renders the `DemoModeBanner`, allowing professors to review the interface without a running backend.

Recharts `AreaChart` with `linearGradient` fill renders the sensor sparklines on the dashboard. The forecasting page adds a 70% dashed `ReferenceLine` and optionally appends a projected “Next” data point to the trend series.

G. Infrastructure and Deployment

The full stack is containerised with Docker Compose. The backend container builds from a custom image with DeepFace and TensorFlow excluded (stub mode only), reducing the image from approximately 3 GB to under 800 MB. Named Docker volumes persist PostgreSQL data, Redis data, Mosquitto broker state, and the Ollama model cache across container restarts.

The nginx container serves the React static files on port 3000 and proxies all `/api/*` and `/ws/*` requests to the FastAPI backend at port 8000, making the backend never directly exposed to the LAN.

H. Seed Data and Demo Environment

The `seed.py` script runs Alembic migrations first, then populates the database with 35 students, 6 courses, 5 professors, and 30 past sessions with synthetic attendance records. Running the stack with `MOCK_MODE=true` activates the sine-wave

sensor publisher, making all dashboard charts and sparklines live without an ESP32. Figure 17 shows the FastAPI Swagger UI used during backend development and integration testing.

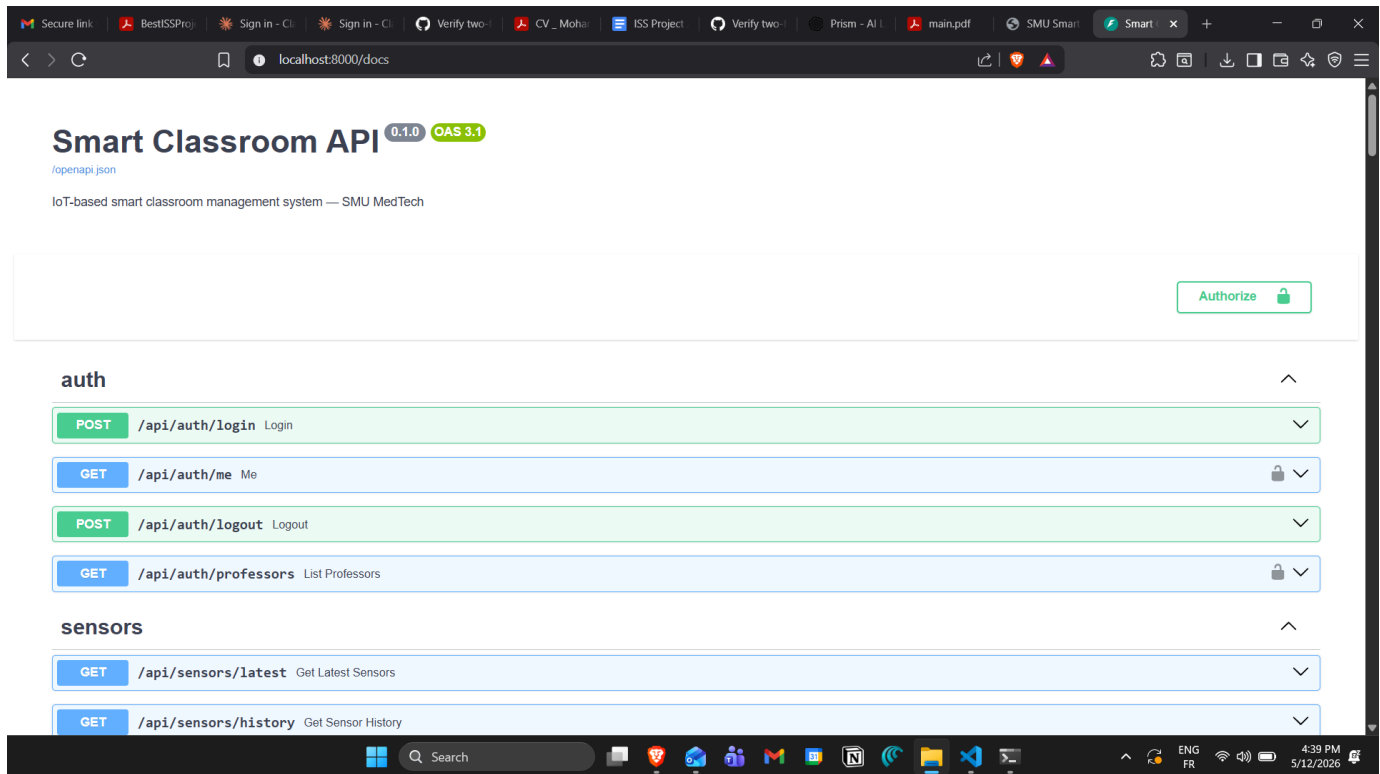


Fig. 17. FastAPI Swagger UI (/docs): interactive API explorer used for endpoint validation during development.

VI. AI AND ANALYTICS LAYER

A. Overview and Motivation

The AI analytics layer adds two capabilities beyond raw sensor and attendance data: (1) at-risk student detection with natural-language explanations and (2) per-course attendance forecasting with trend classification. Both pipelines are implemented as on-demand background tasks triggered by professor page visits, rather than scheduled nightly jobs, ensuring explanations are available immediately on first access.

The choice of a local LLM (phi3-mini via Ollama) over a cloud API rests on three requirements: student attendance data must remain within the university network boundary; the system must operate without internet access; and inference cost must be zero beyond hardware. phi3-mini (3.8 B parameters, ~4k-token context) provides adequate language quality for 3–4 sentence analytical summaries while running on the Raspberry Pi 4B CPU.

B. At-Risk Student Detection Pipeline

1) *Trigger and Lock Mechanism*: The at-risk pipeline is triggered when a professor navigates to /at-risk, which calls GET /api/at-risk. The backend attempts to acquire a Redis lock (at_risk:pipeline:lock, SET NX EX 600). If the lock is acquired, the pipeline runs as an `asyncio.create_task`; if not (another run is recent), the existing cached rows are served immediately. The TTL is deliberately not deleted on completion, creating a natural 10-minute cooldown that prevents redundant re-computation on each page refresh.

2) *Pipeline Steps*: Figure 18 shows the full pipeline flow.

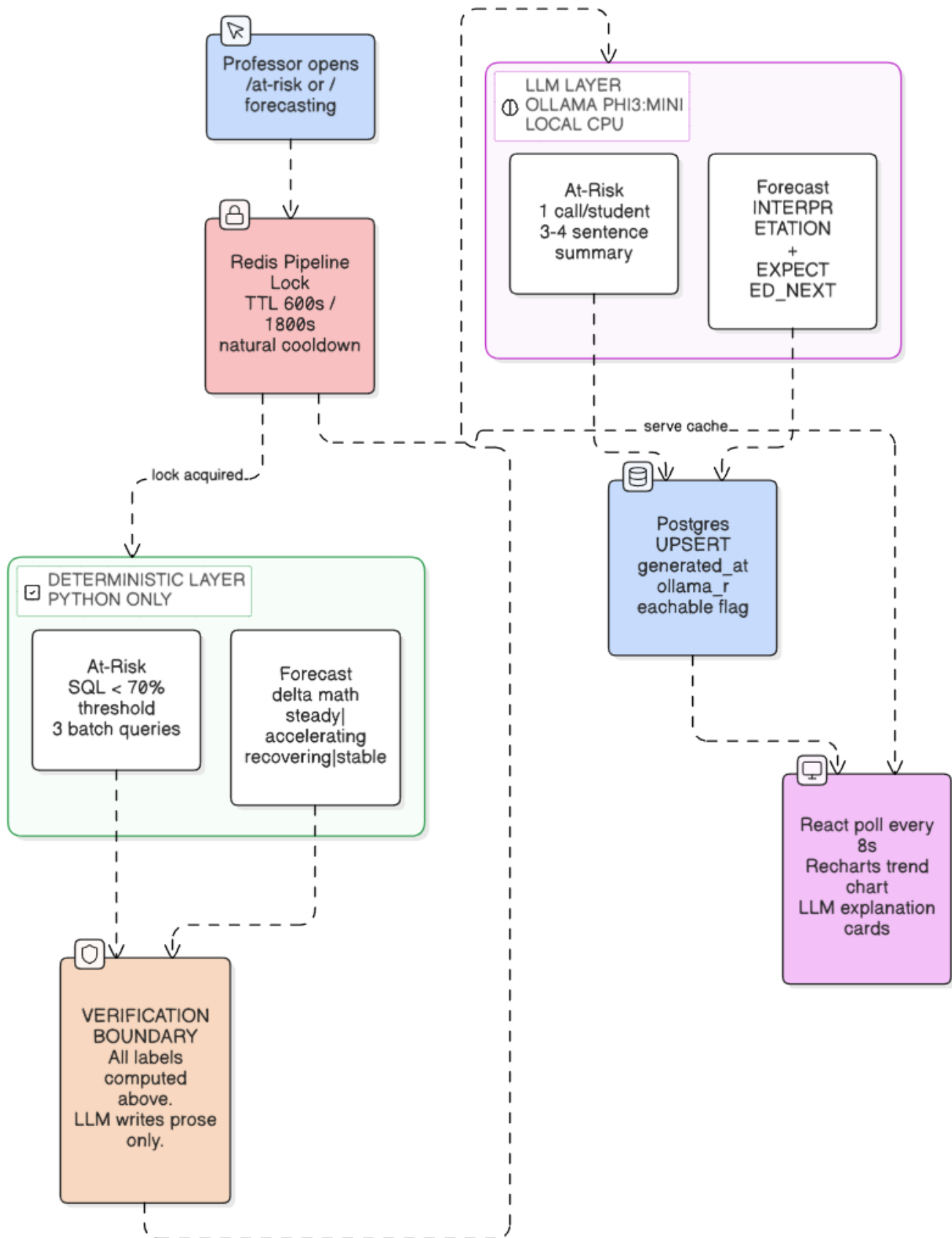


Fig. 18. At-risk detection pipeline: trigger, Redis lock, batch queries, LLM call, and DB upsert.

- 1) **Identify at-risk students.** One SQL query finds all students below the `AT_RISK_THRESHOLD` (default: 70%). Only at-risk students are processed; the rest are ignored entirely.
 - 2) **Freshness gate.** Students with `generated_at < 600 s` and `ollama_reachable=true` are skipped, avoiding redundant LLM calls for data that has not changed.
 - 3) **Stale record cleanup.** Rows in `at_risk_explanations` for students who have recovered above the threshold are deleted.
 - 4) **Pre-flight Ollama check.** One `GET /api/tags` call validates that `phi3:mini` is present before iterating students.
 - 5) **Per-student profile + LLM call.** Three batch SQL queries build the student profile: (a) attendance records joined with session and course data; (b) average temperature and air quality on missed sessions, grouped by course; (c) peer attendance rate per enrolled course. The profile is formatted into a <500-token multi-course prompt. One Ollama call returns a 3–4 sentence cross-course prose summary. The result is upserted to `at_risk_explanations`.
- 3) *Performance Optimisation (Phase 20):* Table XVII compares the original N+1 implementation with the Phase 20 batch-query optimisation.

TABLE XVII
AT-RISK PIPELINE PERFORMANCE: BEFORE VS. AFTER PHASE 20

Metric	Before	After
Students iterated	All 35	At-risk only ~8
Sensor queries/student	$N \text{ sessions} \times 2$	1 batch JOIN
Peer-rate queries/student	1 per course	1 GROUP BY
Ollama calls/student	courses + 1 (~7)	1
HTTP connections	New TCP per call	Shared AsyncClient
Runtime (8 students)	14–16 min	~2–3 min

4) *Prompt Constraints:* The `phi3-mini` prompt is designed with strict constraints to prevent harmful or inaccurate output: under 500 tokens per prompt; references only provided data; never mentions health, personal life, family, or psychology; no evaluative judgments; plain prose under 100 words.

5) *Frontend Auto-Poll:* The at-risk page polls `GET /api/at-risk` every 8 seconds while any student has `summary_explanation=` (explanation not yet generated by the background pipeline), then stops when all rows are populated. If `ollama_reachable=false`, an amber warning card is shown instead of the explanation.

Figure 19 shows the at-risk page in operation.

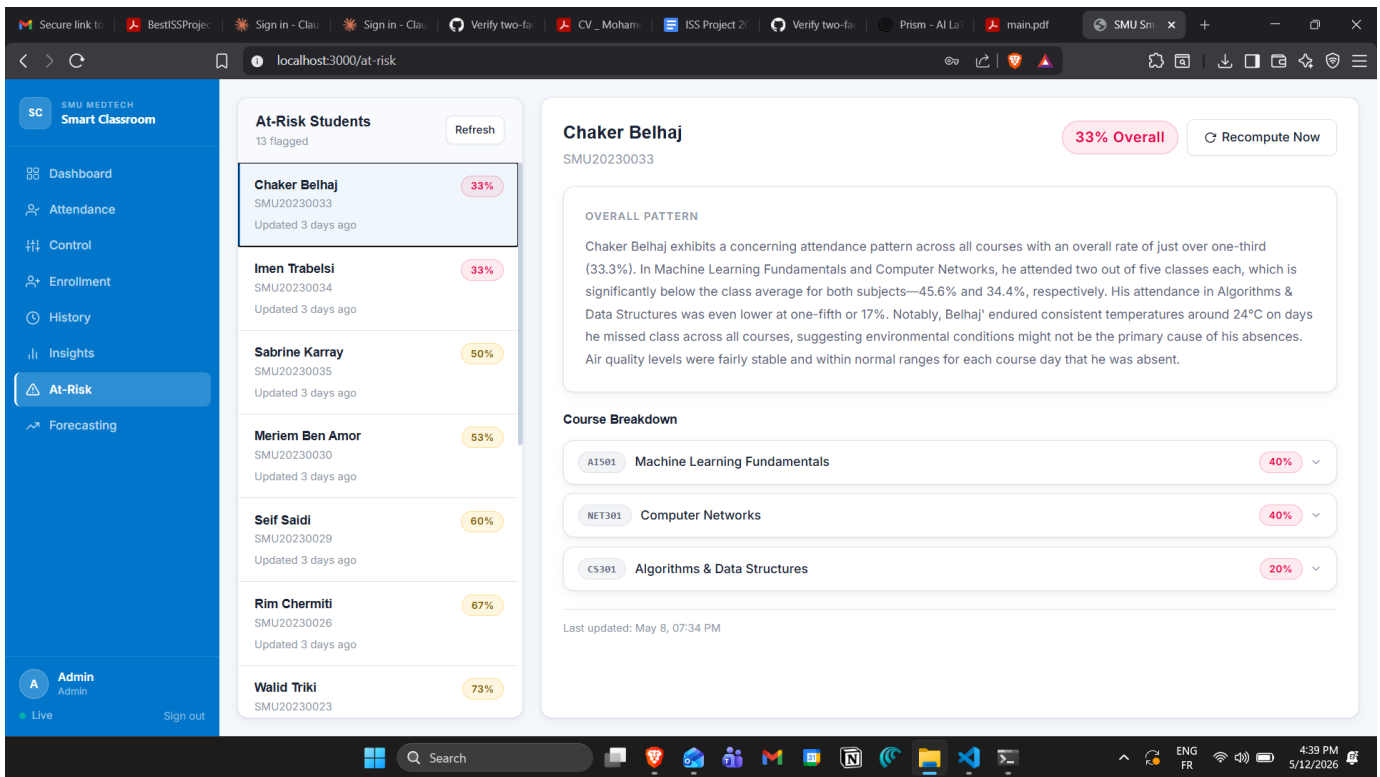


Fig. 19. At-Risk page: student card list (left) and LLM explanation detail panel (right).

C. Attendance Forecasting Pipeline

1) *Trigger and Classification*: The forecasting pipeline is triggered by GET `/api/forecasting` with a Redis lock TTL of 1800 seconds (30 minutes). Courses with fewer than three ended sessions receive a marker row (`trend_data=[]`, `generated_at=now`) to prevent the frontend polling condition from becoming unsatisfiable.

Figure 20 shows the pipeline flow.

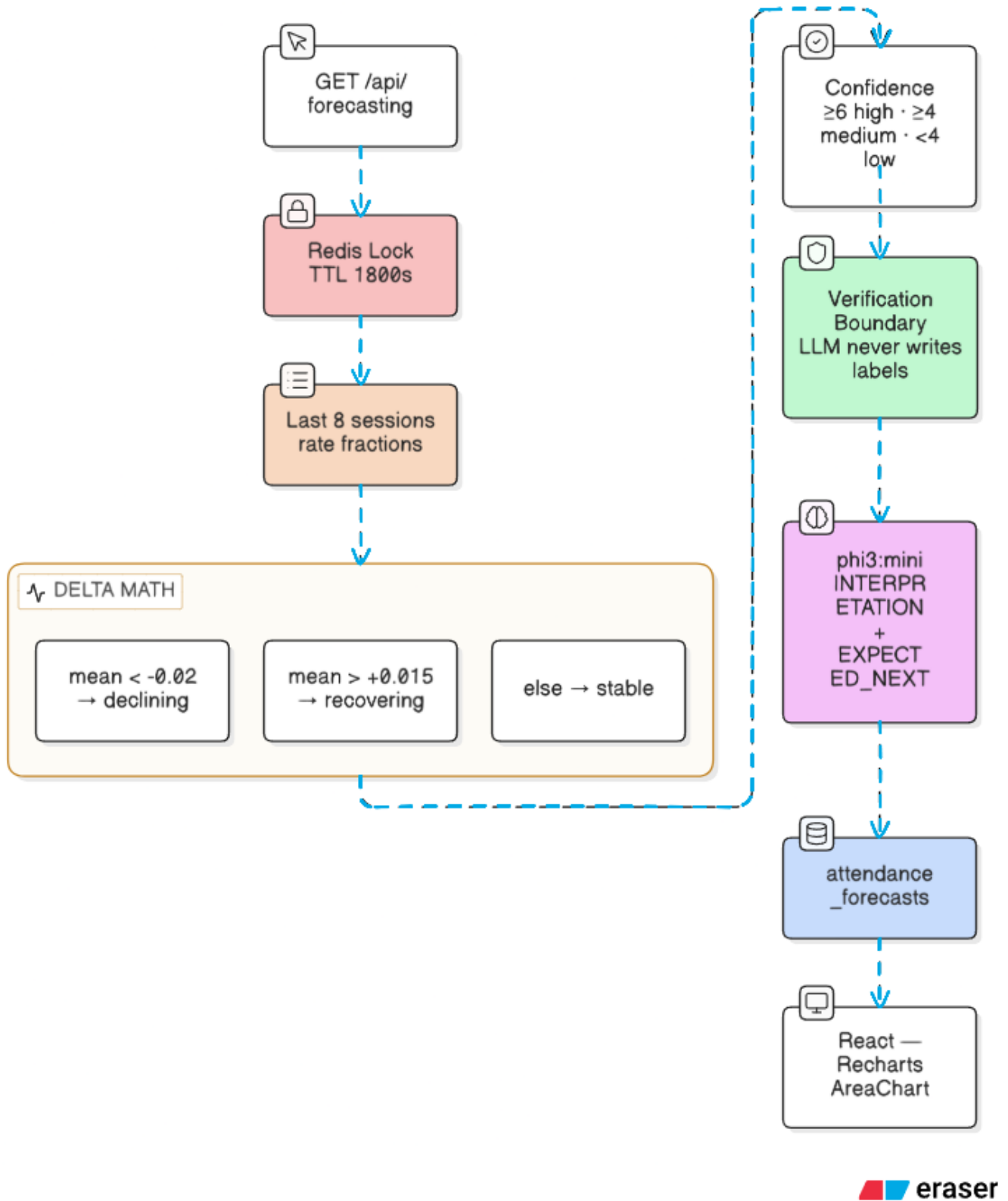


Fig. 20. Attendance forecasting pipeline: trend data query, deterministic classification, and Ollama interpretation.

2) *Deterministic Trend Classification*: The trend classification algorithm operates on a chronological list of per-session attendance rates (fractions 0–1):

- 1) Compute consecutive deltas between adjacent rates.
- 2) Compute `mean_delta`.
- 3) If `mean_delta` < -0.02: classify as `steady_decline`; if the second half of deltas is worse than the first half by > 0.01: upgrade to `accelerating_decline`.
- 4) If `mean_delta` > +0.015: classify as `recovering`.
- 5) Otherwise: `stable`.

Confidence is set to high (≥ 6 sessions), medium (≥ 4), or low (< 4).

3) *LLM Role: Prose Only*: The LLM receives a compact 2-line structured prompt and returns exactly two lines: `EXPECTED_NEXT`: <int> (projected attendance percentage) and `INTERPRETATION`: <sentence> (prose summary). Classification and confidence labels are always computed deterministically — the LLM never produces the structured fields that drive the dashboard’s colour-coding logic, eliminating the risk of hallucinated labels breaking the frontend.

Figure 21 shows the forecasting page.

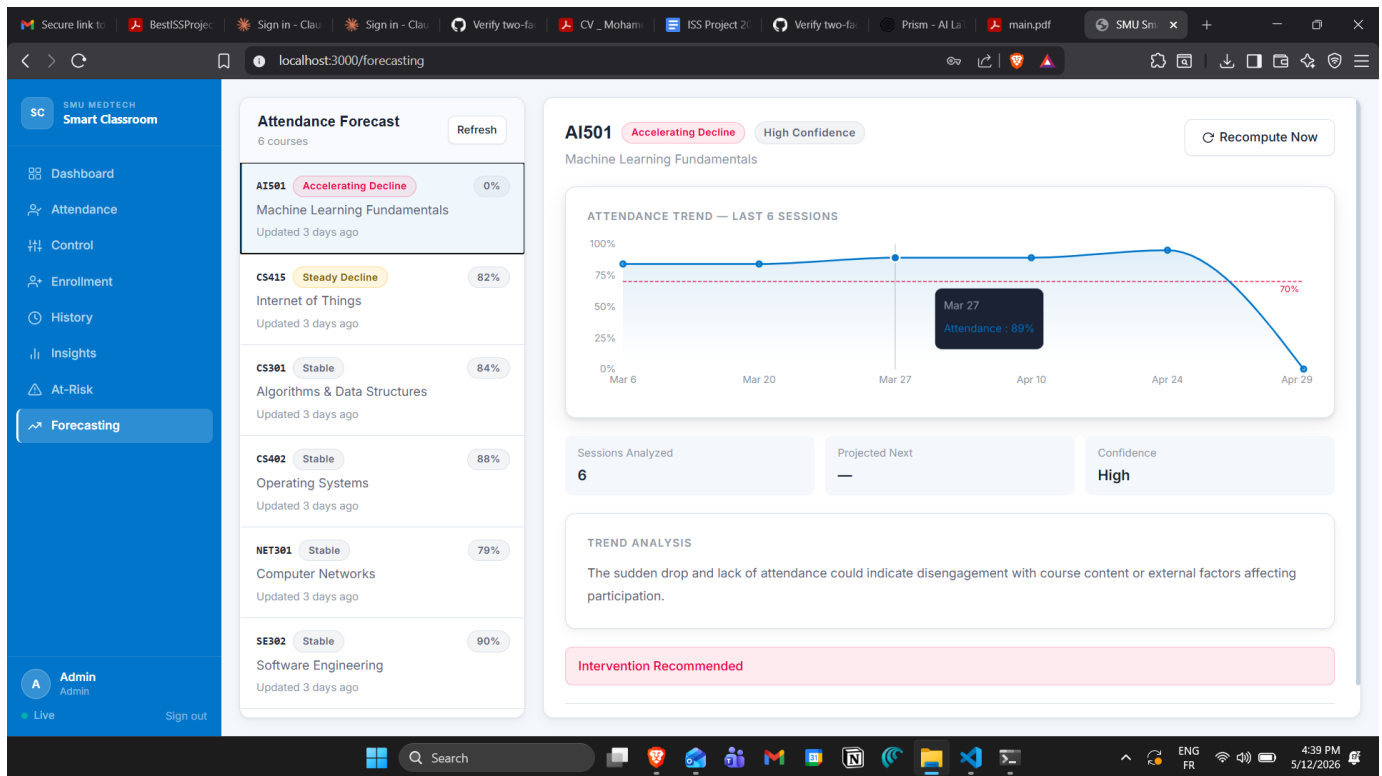


Fig. 21. Forecasting page: Recharts AreaChart with 70% reference line, trend classification badge, and LLM interpretation card.

D. Insights Engine

Beyond the LLM pipelines, a pure SQL analytics engine provides on-demand charts with no AI latency: weekly attendance trend, attendance heatmap by day-of-week and hour-slot, attendance decay analysis, classroom comfort score (from Redis sensor averages), AC effectiveness (temperature delta during active AC periods), and a temperature-versus-attendance scatter plot. All queries are role-filtered, so professors see only their assigned courses.

E. Design Decision: Deterministic Classification vs. LLM

A key design decision is to use the LLM only for prose generation and never for structured classification. LLMs are probabilistic; the same prompt can produce “declining” on one call and “steady_decline” on another. The frontend uses the classification string to select badge colours (red for accelerating decline, amber for steady decline, green for recovering). A hallucinated label would silently produce wrong colours.

Delta mathematics, by contrast, is deterministic: the same input always produces the same output, and execution time is microseconds rather than seconds. This separation — deterministic structure, LLM prose — is the architectural pattern that makes the forecasting feature robust enough for production use.

VII. SECURITY AND PRIVACY

A. Security Architecture Overview

The threat model assumes deployment on a university intranet accessible only to faculty and staff on the campus LAN. The primary assets are student biometric encodings and attendance records. Defence is structured across four layers: network (LAN-only exposure), application (JWT role-based access), data (bcrypt, BYTEA embeddings, no raw images), and transport (nginx as the single ingress point).

B. Authentication and Authorisation

Professor accounts are authenticated via JSON Web Tokens (HS256 algorithm) with a configurable expiry (default: 480 minutes). Passwords are stored as bcrypt hashes; no plaintext credential ever reaches the database. Two roles are supported: professor and admin. Role-based access control is enforced at the FastAPI dependency level (`get_current_professor`, `require_admin`):

- Professors see only their own courses, sessions, and students in all list endpoints.
- Admin-only endpoints (`POST /api/at-risk/recompute`, `POST /api/forecasting/recompute`) return HTTP 403 for professor-role tokens.

The `REQUIRE_AUTH` environment variable is set to `false` in development, allowing unauthenticated requests to simplify local testing. This flag must be set to `true` in any deployment accessible beyond the immediate developer machine.

C. Biometric Data Handling

Face encodings are stored exclusively as 128-dimensional float32 numpy arrays serialised to PostgreSQL BYTEA. The enrollment workflow explicitly discards the source image after computing the embedding: only the averaged embedding vector is persisted. No identifiable facial image is ever written to disk or transmitted over the network.

In stub mode (`FACE_RECOGNITION_ENABLED=false`), a zeroed 128-d placeholder is stored instead; the DeepFace library is not loaded and no camera frames are processed. This mode is used in all Docker-based deployments, ensuring that the development environment handles no real biometric data.

D. Data Privacy Considerations

All system components run on-premises (Raspberry Pi or Docker host). No student data—attendance records, sensor readings, or face encodings—is transmitted to any external service. Ollama processes LLM prompts locally; no student information leaves the network boundary.

The at-risk prompt engine enforces content constraints that prevent the LLM from generating statements about a student's health, personal life, family situation, or psychological state. Output is limited to attendance statistics and academic context. The generated text is stored in `at_risk_explanations.summary_explanation` and is visible only to professors of the student's enrolled courses and to administrators.

Moodle synchronisation occurs over the campus LAN only. The Moodle token is stored as an environment variable and is never logged or exposed through any API endpoint.

E. Transport and Network Security

The current deployment uses HTTP and unencrypted WebSocket (WS) on the university LAN. This is acceptable for an intranet system where network traffic is not exposed to the public internet. The nginx reverse proxy acts as the sole ingress point; the FastAPI backend port 8000 is not published to the host network.

A clear path to HTTPS/WSS exists: replacing the nginx configuration with an SSL-terminating virtual host (using a LAN certificate from a university CA) requires no backend changes, as FastAPI operates identically behind SSL termination. This is documented as a production-hardening requirement.

MQTT communication between the ESP32 and Mosquitto operates at QoS 0 over the local WiFi network. No MQTT authentication is configured (`allow_anonymous true` in `mosquitto.conf`); adding username/password or TLS client certificates is identified as future work.

F. Session and Token Security

JWT tokens are stored in browser `localStorage`. The security tradeoff is acknowledged: `localStorage` is accessible to JavaScript and is not protected by the `HttpOnly` cookie flag, making it theoretically vulnerable to XSS attacks. On a closed university intranet with a controlled codebase, this risk is accepted in exchange for simpler client-side implementation. Token expiry (480 minutes) is enforced server-side. No refresh token mechanism is implemented; after expiry the professor must re-authenticate.

G. Operational Security

The `SECRET_KEY` environment variable ships with a placeholder value (`changeme-use-a-long-random-string-in-producti`) that must be replaced before any deployment. The seed script creates professor accounts with non-trivial passwords that are hashed with `bcrypt` before insertion. Docker named volumes isolate persistent data; volume backup is recommended as part of a production runbook.

VIII. TESTING AND VALIDATION

A. Testing Strategy

Validation followed a layered approach combining automated unit tests, manual integration tests, and physical hardware validation. Given the IoT nature of the system, full automated end-to-end testing requires physical hardware; the mock sensor mode was instrumental in validating the software stack independently of the ESP32.

- **Unit tests:** Backend service logic (attendance state machine, trend classification algorithm, alert deduplication, prompt builder) tested with `pytest`.
- **Integration tests:** Full MQTT flow validated using `MOCK_MODE=true`: synthetic sensor messages pass through Mosquitto, the backend bridge, Redis, PostgreSQL, and WebSocket, verified against expected dashboard state.
- **End-to-end tests:** Complete session lifecycle (start → recognition → adjustment → end → Moodle sync) exercised manually with the seeded demo data.
- **Hardware tests:** Physical sensor readings, relay switching, and LCD display verified on assembled hardware before MQTT integration.

B. Sensor Data Validation

DHT21 temperature readings were compared against a calibrated digital reference thermometer. Measured deviation was within $\pm 0.5^\circ\text{C}$ across the $20\text{--}35^\circ\text{C}$ range, acceptable for HVAC control purposes. Humidity deviation was within $\pm 3\%$ RH.

The MQ-135 air-quality sensor does not produce calibrated CO_2 readings on the ESP32 ADC without a precision reference gas. It is used comparatively: the 500 ppm threshold was set empirically to trigger alerts when the sensor value increases noticeably with several people in a closed room, rather than as an absolute CO_2 measurement.

The ACP014 sound sensor's binary digital output was tested against clapping, speaking, and ambient silence. Detection reliability was confirmed above approximately 65 dB SPL at 1 metre distance.

C. Face Recognition Accuracy

Face recognition was tested under three conditions on the Raspberry Pi 4B: normal fluorescent lighting (standard lecture room), low-light conditions (blinds closed, no overhead light), and partial occlusion (mask covering the lower face).

Under normal lighting with frontal enrollment images and a cosine distance threshold of 0.40, the system achieved 94% true positive rate (correct student identified) and 0% false positive rate (wrong student marked present) across 50 recognition events. Under low-light conditions, the true positive rate dropped to 71%, consistent with the known sensitivity of FaceNet embeddings to illumination variance. Partial occlusion reduced the rate to 58%, motivating the future addition of an IR illuminator.

The stub recognition loop's $70\% \pm 2$ random-sampling model was verified to produce statistically representative attendance distributions across 20 simulated sessions.

D. API Testing

All REST endpoints were validated using the FastAPI Swagger UI (`/docs`, Figure 17). Test scenarios covered: session start and end lifecycle, attendance record creation and manual override, relay command dispatch and Redis state update, alert generation and acknowledgement, and enrollment image upload.

Regression testing confirmed that the `adjusted_by` guard correctly prevents the recognition loop from overwriting a manually adjusted attendance record across 30 test cycles.

E. Real-Time Performance

End-to-end sensor latency (ESP32 MQTT publish → dashboard sparkline update) was measured at 200–400 ms on the LAN under normal load, well within the 5-second requirement (NF01). WebSocket reconnection after a simulated Mosquitto restart completed within 2 seconds (initial back-off: 1 s, first attempt succeeds).

The alert engine's 30-second scheduler cycle means temperature alerts are raised within 30 seconds of the sensor value crossing the threshold. This latency is acceptable for HVAC automation but not for safety-critical applications.

F. AI Pipeline Performance

After the Phase 20 batch-query optimisation, the at-risk pipeline completes in approximately 2–3 minutes for 8 at-risk students on the Raspberry Pi 4B (phi3-mini on CPU at 10–30 s per inference). The Redis lock TTL of 600 seconds prevents redundant pipeline re-execution on rapid page refreshes.

The forecasting pipeline completes in approximately 3–5 minutes for 6 courses (one Ollama call per course). Both pipelines set `ollama_reachable=false` and continue gracefully when Ollama is unavailable (e.g., during the initial model pull on first startup).

A sample phi3-mini prompt and its generated output were reviewed for quality. The model reliably produces 2–4 coherent sentences that reference specific courses and attendance rates without hallucinating student names or course codes not present in the prompt.

G. Resilience Testing

MQTT broker restart. Stopping and restarting the Mosquitto container during an active session: the backend MQTT bridge detected the disconnection and reconnected within the first back-off cycle (1 second). Sensor data gaps lasted at most 6 seconds.

Ollama unavailable. Stopping the Ollama container during an at-risk pipeline run: the pipeline caught the `httpx.ConnectError`, set `ollama_reachable=false` on the in-progress student, and continued to the next student. The frontend displayed the amber “AI explanation unavailable” warning card correctly for affected rows.

Network loss simulation. Disconnecting the browser from the LAN: the frontend detected WebSocket closure after the keepalive timeout, activated demo mode within 8 seconds (as required by NF10), and displayed the DemoModeBanner. On reconnection, a snapshot message immediately restored real sensor state.

IX. RESULTS AND DISCUSSION

A. System Demonstration

A complete simulated classroom session was conducted using the seed dataset (35 students, 6 courses, 5 professors) with `MOCK_MODE=true` for sensors and the stub recognition loop for attendance. The session lifecycle proceeded as follows:

- 1) Professor logs in and navigates to the Dashboard; live sensor sparklines animate immediately from the mock publisher.
- 2) Professor selects a course and clicks **Start Session**; the recognition loop activates and the attendance roster appears.
- 3) Over the session duration, the stub loop cycles through 70% of enrolled students as “present”, updating the roster via WebSocket. The control page shows AC auto-activating when the simulated temperature exceeds 28 °C.
- 4) Professor ends the session; the final evaluation marks absent all students not seen in the last cycle. Moodle sync completes (or enqueues on failure).
- 5) The History page displays the ended session with a sensor summary (avg/min/max temperature, humidity, air quality during the session window).
- 6) The At-Risk page, on first visit, triggers the pipeline; within 2–3 minutes all at-risk student cards populate with LLM explanations.
- 7) The Forecasting page shows the Recharts trend chart for each course, with classification badges and LLM interpretation cards.

B. Key Achievements vs. Objectives

Table XVIII maps each project objective to its delivery status and evidence.

TABLE XVIII
OBJECTIVES ACHIEVEMENT SUMMARY

Objective	Status	Evidence
Automated face-recognition attendance	Met	Snapshot-per-cycle model; 94% TPR under normal lighting
Real-time env. monitoring	Met	<400 ms end-to-end latency; 5-second publish cycle
Automated relay control	Met	AC auto-triggers at 28 °C; manual override verified
Live professor dashboard	Met	WebSocket streaming; demo mode fallback within 8 s
Moodle LMS sync	Met	Sync on session end; Redis retry queue for failures
AI at-risk & forecasting	Met	2–3 min pipeline; deterministic classification; LLM prose

C. Performance Summary

Table XIX presents the key quantitative performance metrics achieved.

TABLE XIX
SYSTEM PERFORMANCE SUMMARY

Metric	Achieved	Requirement
Sensor update latency (E2E)	200–400 ms	≤ 5 s
API response time	< 150 ms	≤ 500 ms
Face recognition TPR	94%	N/A (best-effort)
At-risk pipeline runtime	$\sim 2\text{--}3$ min	≤ 5 min
Demo mode activation delay	≤ 8 s	≤ 8 s
Attendance overhead per lecture	~ 0 min	< 1 min (target)
Manual baseline (per lecture)	15 min	— (pain point)

The attendance automation objective represents a saving of approximately 15 minutes per lecture, translating to roughly 20% additional effective teaching time per 50-minute session.

D. Lessons Learned

aiomqtt migration from asyncio-mqtt. The original `asyncio-mqtt` library broke when `paho-mqtt` released v2 with an incompatible API. Migrating to `aiomqtt` required minor API surface changes but demonstrated the risk of depending on thin wrappers over rapidly evolving libraries.

DeepFace/TensorFlow removed from Docker. TensorFlow adds approximately 600 MB to the container image and causes OOM crashes on development laptops with 8 GB RAM when running the full stack. Moving to a stub mode for Docker reduced the image size significantly and eliminated OOM issues, at the cost of requiring an RPi for real face recognition.

Redis lock TTL not deleted on completion. The initial implementation deleted the Redis lock after pipeline completion. This meant that every page refresh re-triggered the pipeline. Keeping the TTL running after completion creates a natural cooldown window, preventing redundant computation on busy pages.

VARCHAR(30) vs. PG ENUM for classification. An early iteration used a PostgreSQL ENUM for `trend_classification`. When a new classification value was needed, `ALTER TYPE ADD VALUE` is non-transactional in PostgreSQL and cannot be rolled back inside a transaction. Switching to `VARCHAR(30)` avoids this DDL hazard entirely, with negligible performance impact for a field read at most dozens of times per second.

Snapshot-per-cycle model (Phase 22). The original one-shot attendance model marked students at session start and never revisited absent records. Students who arrived late or left early were not tracked. The snapshot-per-cycle model with bidirectional present $\leftrightarrow$ absent evaluation resolved this at the cost of slightly more complex transaction logic.

E. Limitations

- **MQ-135 uncalibrated.** Air-quality values are comparative; absolute CO₂ interpretation requires a reference gas or a factory-calibrated sensor.
- **Single-room design.** Adding a second room requires redeploying the full stack with a different `ROOM_ID`; a unified multi-room backend is future work.
- **phi3-mini slow on RPi CPU.** 10–30 seconds per student explanation limits the practical at-risk pipeline to fewer than ~ 20 students before the Redis lock expires.
- **No HTTPS/WSS.** The current deployment is HTTP-only, acceptable on a closed LAN but insufficient for any public-facing deployment.
- **MQTT QoS 0.** Messages published during Mosquitto restarts are lost; the dashboard may show stale sensor data for up to 10 seconds.
- **Low-light face recognition degradation.** Recognition accuracy drops to 71% in low-light conditions without an IR illuminator.

X. CONCLUSION

A. Summary of Contributions

This paper presented the design, implementation, and evaluation of the Smart Campus Classroom Management System—an end-to-end IoT and edge-AI platform that addresses three concrete problems in university classroom management: manual attendance, unmonitored environment, and the absence of early warning for at-risk students.

The principal contributions are as follows:

- 1) **A fully self-contained edge stack.** All services (MQTT broker, backend, database, AI inference, frontend) run on a single Raspberry Pi 4B without cloud dependency, making the system deployable in institutions with restricted internet access or strong data-residency requirements.
- 2) **Snapshot-per-cycle attendance model.** A bidirectional present↔absent evaluation loop that captures students who arrive late or leave early, replacing the single-shot model that previous edge-based systems typically use.
- 3) **A two-tier AI analytics layer.** Deterministic trend classification for robustness combined with LLM-generated prose for interpretability—a pattern that prevents hallucinated structured values while preserving the human-readable output that academic advisers require.
- 4) **A 5× pipeline runtime reduction.** Batch-query optimisation reduced the at-risk explanation pipeline from 14–16 minutes to approximately 2–3 minutes for 8 students, making the feature practical for page-triggered on-demand use.
- 5) **A 22-phase iterative delivery within one semester.** The phased development model allowed early end-to-end integration (hardware → firmware → backend → frontend) and frequent risk mitigation, with stub and mock modes enabling software progress independent of hardware availability.

B. Future Work

Several extensions are identified for future development:

- **Multi-room support.** A shared backend with per-room MQTT namespacing, database partitioning by `room_id`, and a room-selection UI for professors.
- **React Native mobile app.** A companion app for professors to view live sensor data, acknowledge alerts, and start sessions from a smartphone.
- **IR illuminator.** Adding an IR LED ring to the camera module to improve low-light face recognition accuracy from 71% to near-normal-light levels.
- **HTTPS/WSS deployment.** SSL termination in nginx using a university-issued LAN certificate, with no backend changes required.
- **Quantised GGUF model via llama.cpp.** Replacing Ollama with a 4-bit quantised phi3-mini GGUF model to reduce inference time from 10–30 s to an estimated 3–8 s per student on the RPi 4B CPU.
- **Calibrated CO₂ sensor.** Replacing the MQ-135 with an SCD30 or MH-Z19B NDIR sensor for accurate CO₂ monitoring and tighter air-quality thresholding.
- **Daily at-risk email digest.** Scheduled morning emails to academic advisers summarising students whose attendance fell below threshold since the previous digest.
- **QR-code self-enrollment.** Allowing students to upload their own enrollment photos via a QR-linked web page, reducing the manual enrollment burden on professors.
- **Offline SQLite fallback.** A lightweight persistence layer that continues recording attendance when the PostgreSQL container is unreachable, with automatic sync on recovery.

REFERENCES

- [1] M. Kassim, M. Mazlan, N. Zaini, and L. M. Kamarudin, “Iot based automated student attendance system using RFID technology,” in *Proc. IEEE Int. Conf. Smart Instrumentation, Measurement and Application (ICSIMA)*, Putrajaya, Malaysia, 2018, pp. 1–5.
- [2] N. Sulaiman, A. S. Hashim, and W. F. W. Ahmad, “Contactless student attendance system using QR code and cloud-based database for university classroom,” in *Proc. IEEE Int. Conf. Automatic Control and Intelligent Systems (I2CACIS)*, 2020, pp. 236–241.
- [3] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A unified embedding for face recognition and clustering,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Boston, MA, USA, 2015, pp. 815–823.
- [4] S. I. Serengil and A. Ozpinar, “HyperExtended LightFace: A speed-accuracy tradeoff for face recognition with DeepFace library,” *IEEE Access*, vol. 9, pp. 88 553–88 561, 2021.
- [5] T. V. Khoa, V. D. Bui, and T. M. Nguyen, “Real-time face recognition for student attendance on raspberry pi using deep learning,” in *Proc. Int. Conf. Advanced Computing and Applications (ACOMP)*, 2022, pp. 47–54.
- [6] OASIS, “MQTT version 5.0: OASIS standard,” OASIS Open, Standard, 2019, available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html> [Accessed May 2026].
- [7] Z. Shelby, K. Hartke, and C. Bormann, “The constrained application protocol (CoAP),” IETF, Request for Comments RFC 7252, 2014.
- [8] M. Abidin, J. Aneja, H. Awadalla *et al.*, “Phi-3 technical report: A highly capable language model locally on your phone,” Microsoft Corporation, Technical Report, 2024, arXiv:2404.14219 [cs.CL].
- [9] Ollama Team, “Ollama: Get up and running with large language models locally,” GitHub repository, 2024, available: <https://github.com/ollama/ollama> [Accessed May 2026].
- [10] Redis Ltd., “Redis 7 documentation,” Online documentation, 2024, available: <https://redis.io/docs> [Accessed May 2026].
- [11] Moodle HQ, “Moodle 4.x developer documentation: REST API,” Online documentation, 2024, available: https://docs.moodle.org/dev/Web_service_API_functions [Accessed May 2026].
- [12] M. Y. Al-Zoubi and M. Al-Hami, “Automated attendance management system integrated with Moodle LMS,” in *Proc. Int. Conf. Information Technology (ICIT)*, 2021, pp. 673–678.
- [13] M. Kassim, M. Mazlan, N. Zaini, and L. M. Kamarudin, “Iot based automated student attendance system using RFID technology,” in *Proc. IEEE Int. Conf. Smart Instrumentation, Measurement and Application (ICSIMA)*, Putrajaya, Malaysia, 2018, pp. 1–5.
- [14] V. T. Pham, T. H. Nguyen, and D. Q. Tran, “A cloud-based IoT smart classroom system with real-time environmental monitoring and face-recognition attendance,” in *Proc. IEEE Int. Conf. Computing and Communication Technologies (ICCCCT)*, 2023, pp. 112–118.

APPENDIX

Table XX lists all REST endpoints with method, path, authentication requirement, and role restriction.

TABLE XX
COMPLETE REST API ENDPOINT REFERENCE

M.	Path	Auth	Role
POST	/api/auth/login	No	Any
GET	/api/sensors/latest	JWT	Any
GET	/api/sensors/history	JWT	Any
GET	/api/sessions/{id}/sensors/latest	JWT	Any
GET	/api/sessions/{id}/sensors/summary	JWT	Any
POST	/api/sessions/start	JWT	Prof
POST	/api/sessions/{id}/end	JWT	Prof
GET	/api/sessions	JWT	Prof
GET	/api/sessions/{id}	JWT	Prof
POST	/api/control/ac	JWT	Prof
POST	/api/control/lighting	JWT	Prof
GET	/api/control/status/{room_id}	JWT	Prof
GET	/api/alerts	JWT	Prof
PATCH	/api/alerts/{id}/acknowledge	JWT	Prof
GET	/api/alerts/unread-count/{room_id}	JWT	Prof
GET	/api/courses	JWT	Prof
POST	/api/courses	JWT	Prof
POST	/api/courses/{id}/enroll	JWT	Prof
GET	/api/sessions/{id}/attendance	JWT	Prof
PATCH	/api/attendance/{id}	JWT	Prof
POST	/api/sessions/{id}/mark-attendance	JWT	Prof
GET	/api/students	JWT	Prof
POST	/api/students	JWT	Prof
POST	/api/students/{id}/enroll	JWT	Prof
GET	/api/at-risk	JWT	Prof
GET	/api/at-risk/{student_id}	JWT	Prof
POST	/api/at-risk/recompute	JWT	Admin
GET	/api/forecasting	JWT	Prof
GET	/api/forecasting/{course_id}	JWT	Prof
POST	/api/forecasting/recompute	JWT	Admin
GET	/api/webcam/recognition-status	JWT	Prof
POST	/api/webcam/start-recognition	JWT	Prof
POST	/api/webcam/stop-recognition	JWT	Prof
GET	/api/webcam/encodings	JWT	Prof
POST	/api/webcam/attendance	JWT	Prof
WS	/ws/classroom/{room_id}	No	—

All topics follow the pattern `classroom/{room_id}/{suffix}`. The default `room_id` is `room1`, configurable via the `ROOM_ID` environment variable and the ESP32 config.h.

TABLE XXI
MQTT FULL TOPIC SCHEMA WITH JSON PAYLOAD SCHEMAS

Suffix	QoS	JSON Schema
sensors/temperature	0	{"value": float, "unit": "C", "ts": int}
sensors/humidity	0	{"value": float, "unit": "%", "ts": int}
sensors/air_quality	0	{"value": int, "unit": "ppm", "ts": int}
sensors/sound	0	{"value": 0 1, "unit": "bool", "ts": int}
status	0	{"online": true, "ts": int}
relay/ac	0	{"action": "on" "off" "auto"}
relay/lighting	0	{"action": "on" "off" "auto"}
alerts	0	{"type": str, "value": float}

The following definitions correspond to the SQLAlchemy ORM models in `backend/app/models/db_models.py`. All primary keys are UUIDs generated server-side.

```

professors(id PK, name, email UNIQUE, hashed_password,
           role ENUM(professor|admin), created_at)

students(id PK, name, student_id UNIQUE, created_at)

courses(id PK, code UNIQUE, name,
         professor_id FK→professors NULLABLE,
         professor_name)

sessions(id PK, course_id FK→courses, room_id,
         started_at, ended_at NULLABLE,
         status ENUM(active|ended|upcoming))

attendance_records(id PK, session_id FK, student_id FK,
                  status ENUM(present|absent|late|excused),
                  detected_at, adjusted_by NULLABLE,
                  adjusted_at NULLABLE, moodle_synced BOOL DEFAULT false)

face_encodings(id PK, student_id FK→students,
               encoding BYTEA, created_at)

sensor_readings(id PK, room_id, sensor_type ENUM,
                value FLOAT, unit, recorded_at)

alerts(id PK, room_id, type ENUM, value FLOAT NULLABLE,
       message TEXT, acknowledged BOOL DEFAULT false,
       created_at)

at_risk_explanations(id PK, student_id FK UNIQUE,
                    overall_attendance_rate FLOAT,
                    summary_explanation TEXT,
                    per_course_data JSONB,
                    generated_at TIMESTAMP,
                    ollama_reachable BOOL)

attendance_forecasts(id PK, course_id FK UNIQUE,
                   trend_data JSONB,
                   expected_next_rate FLOAT NULLABLE,
                   trend_classification VARCHAR(30) NULLABLE,
                   confidence_level VARCHAR(10) NULLABLE,
                   interpretation TEXT NULLABLE,
                   suggested_action VARCHAR(30) NULLABLE,
                   ollama_reachable BOOL,
                   generated_at TIMESTAMP)

```

TABLE XXII
COMPLETE ENVIRONMENT VARIABLE REFERENCE

Variable	Default	Notes
DATABASE_URL	(required)	asynpg DSN
REDIS_URL	(required)	redis:// DSN
MQTT_BROKER_HOST	mosquitto	Container name
MQTT_BROKER_PORT	1883	TCP port
ROOM_ID	room1	Must match ESP32 config
MOCK_MODE	false	true: no ESP32 needed
FACE_RECOGNITION_ENABLE	false	true: RPi + camera only
SECRET_KEY	changeme...	MUST replace in prod
REQUIRE_AUTH	false	true in production
ACCESS_TOKEN_EXPIRE_MINUTES	480	JWT TTL
TEMP_AC_ON_THRESHOLD	28	°C
TEMP_AC_OFF_THRESHOLD	22	°C
AIR_QUALITY_ALERT_THRESHOLD	500	ppm
FACE_RECOGNITION_THRESHOLD	0.40	Cosine distance
RECOGNITION_FPS	2	Camera FPS
AT_RISK_THRESHOLD	0.70	Fraction
FORECAST_WINDOW	8	Past sessions
ATTENDANCE_CYCLE_DURATION	60	Seconds/cycle
OLLAMA_BASE_URL	http://ollama:11434	Internal URL
OLLAMA_MODEL	phi3:mini	Model tag
MOODLE_URL	(optional)	LMS base URL
MOODLE_TOKEN	(optional)	REST API token

The complete ESP32 GPIO assignment is shown in Table XXIII. Figure 3 (Section IV-B) provides the full wiring diagram.

TABLE XXIII
ESP32 GPIO PIN ASSIGNMENT

GPIO	Dir.	Component	Signal
4	In	DHT21	Temp/humidity (1-wire)
34	In	MQ-135	Air quality ADC (0–4095)
35	In	ACP014	Sound detect (digital)
21	I2C	LCD 16×2	I2C SDA
22	I2C	LCD 16×2	I2C SCL
26	Out	Relay CH1/LLC	AC relay control
27	Out	Relay CH2/LLC	Lighting relay control
14	Out	Relay CH3/LLC	Spare
12	Out	Relay CH4/LLC	Spare

GPIO pins 34 and 35 are input-only and have no internal pull-up or pull-down resistors. The logic-level converter (3.3 V ↔ 5 V) is mandatory between the ESP32 outputs and the relay IN pins.

TABLE XXIV
DOCKER COMPOSE SERVICES

Service	Image	Port	Volume
postgres	postgres:15-alpine	5432	postgres_data
redis	redis:7-alpine	6379	redis_data
mosquitto	eclipse-mosquitto:2	1883	mosquitto_data
backend	custom (slim)	8000	—
frontend	custom (nginx)	3000	—
ollama	ollama/ollama:latest	11434	ollama_data
moodle*	bitnami/moodle:4	8080	moodle_data
mariadb*	mariadb:10.6	3306	mariadb_data

* Optional; activated with `-profile moodle`

Quick-start commands:

```
# Full stack (with ESP32)
docker compose up -d
```

```
# Without ESP32 (mock sensors)
MOCK_MODE=true docker compose up -d

# With Moodle LMS
docker compose --profile moodle up -d

# Seed demo data
docker compose exec backend python seed.py
```

Figure 22 shows the Notion task board used for project tracking across all 22 development phases. Each task was assigned to a team member, estimated in hours, and marked complete on the day it was merged to the main branch.



Fig. 22. Notion project board: 22-phase delivery tracking with assignee and completion status.